

```
In [2]: # Import required libraries

import obspy
from obspy import UTCDateTime

import numpy as np
import matplotlib.pyplot as plt

from pathlib import Path
from collections import Counter
```

```
In [4]: # Folder containing the seismic files

folder = Path(r"C:\Users\marwan\Downloads\Strokkur_data_2026-
extracted\Strokkur_data_2026")
```

```
In [5]: stream = obspy.Stream()

for file in sorted(folder.iterdir()):

    if not file.is_file():
        continue

    if file.name == ".DS_Store":
        continue

    try:
        stream += obspy.read(str(file))
        print(f"Loaded: {file.name}")

    except Exception as error:
        print(f"Skipped: {file.name}")
        print(error)
```

```
Loaded: 2Z.GE2..HHE.D.2026.001
Loaded: 2Z.GE2..HHE.D.2026.002
Loaded: 2Z.GE2..HHE.D.2026.003
Loaded: 2Z.GE2..HHE.D.2026.004
Loaded: 2Z.GE2..HHE.D.2026.005
Loaded: 2Z.GE2..HHE.D.2026.006
Loaded: 2Z.GE2..HHE.D.2026.007
Loaded: 2Z.GE2..HHN.D.2026.001
Loaded: 2Z.GE2..HHN.D.2026.002
Loaded: 2Z.GE2..HHN.D.2026.003
Loaded: 2Z.GE2..HHN.D.2026.004
Loaded: 2Z.GE2..HHN.D.2026.005
Loaded: 2Z.GE2..HHN.D.2026.006
Loaded: 2Z.GE2..HHN.D.2026.007
Loaded: 2Z.GE2..HHZ.D.2026.001
Loaded: 2Z.GE2..HHZ.D.2026.002
Loaded: 2Z.GE2..HHZ.D.2026.003
Loaded: 2Z.GE2..HHZ.D.2026.004
Loaded: 2Z.GE2..HHZ.D.2026.005
Loaded: 2Z.GE2..HHZ.D.2026.006
Loaded: 2Z.GE2..HHZ.D.2026.007
Loaded: 2Z.GE4..HHE.D.2026.001
Loaded: 2Z.GE4..HHE.D.2026.002
Loaded: 2Z.GE4..HHE.D.2026.003
Loaded: 2Z.GE4..HHE.D.2026.004
Loaded: 2Z.GE4..HHE.D.2026.005
Loaded: 2Z.GE4..HHE.D.2026.006
Loaded: 2Z.GE4..HHE.D.2026.007
Loaded: 2Z.GE4..HHN.D.2026.001
Loaded: 2Z.GE4..HHN.D.2026.002
Loaded: 2Z.GE4..HHN.D.2026.003
Loaded: 2Z.GE4..HHN.D.2026.004
Loaded: 2Z.GE4..HHN.D.2026.005
Loaded: 2Z.GE4..HHN.D.2026.006
Loaded: 2Z.GE4..HHN.D.2026.007
```

```

Loaded: 2Z.GE4..HHZ.D.2026.001
Loaded: 2Z.GE4..HHZ.D.2026.002
Loaded: 2Z.GE4..HHZ.D.2026.003
Loaded: 2Z.GE4..HHZ.D.2026.004
Loaded: 2Z.GE4..HHZ.D.2026.005
Loaded: 2Z.GE4..HHZ.D.2026.006
Loaded: 2Z.GE4..HHZ.D.2026.007
Loaded: 2Z.GE5..HHE.D.2026.001
Loaded: 2Z.GE5..HHE.D.2026.002
Loaded: 2Z.GE5..HHE.D.2026.003
Loaded: 2Z.GE5..HHE.D.2026.004
Loaded: 2Z.GE5..HHE.D.2026.005
Loaded: 2Z.GE5..HHE.D.2026.006
Loaded: 2Z.GE5..HHE.D.2026.007
Loaded: 2Z.GE5..HHN.D.2026.001
Loaded: 2Z.GE5..HHN.D.2026.002
Loaded: 2Z.GE5..HHN.D.2026.003
Loaded: 2Z.GE5..HHN.D.2026.004
Loaded: 2Z.GE5..HHN.D.2026.005
Loaded: 2Z.GE5..HHN.D.2026.006
Loaded: 2Z.GE5..HHN.D.2026.007
Loaded: 2Z.GE5..HHZ.D.2026.001
Loaded: 2Z.GE5..HHZ.D.2026.002
Loaded: 2Z.GE5..HHZ.D.2026.003
Loaded: 2Z.GE5..HHZ.D.2026.004
Loaded: 2Z.GE5..HHZ.D.2026.005
Loaded: 2Z.GE5..HHZ.D.2026.006
Loaded: 2Z.GE5..HHZ.D.2026.007

```

In [6]: `print(stream)`

63 Trace(s) in Stream:

```

2Z.GE2..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples

```

...

(61 other traces)

...

```

2Z.GE5..HHZ | 2026-01-07T00:00:00.000000Z - 2026-01-
07T13:00:20.870000Z | 100.0 Hz, 4682088 samples

```

[Use "print(Stream.__str__(extended=True))" to print all Traces]

3. Dataset Inspection

This section examines the available stations, channels, sampling rates, and trace metadata before visualizing or processing the seismic data.

```
In [5]: print(stream.__str__(extended=True))
```

```
63 Trace(s) in Stream:
2Z.GE2..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-02T00:00:00.000000Z - 2026-01-
02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-03T00:00:00.000000Z - 2026-01-
03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-04T00:00:00.000000Z - 2026-01-
04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-05T00:00:00.000000Z - 2026-01-
05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-06T00:00:00.000000Z - 2026-01-
06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-07T00:00:00.000000Z - 2026-01-
07T12:08:56.110000Z | 100.0 Hz, 4373612 samples
2Z.GE2..HHN | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHN | 2026-01-02T00:00:00.000000Z - 2026-01-
02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHN | 2026-01-03T00:00:00.000000Z - 2026-01-
03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHN | 2026-01-04T00:00:00.000000Z - 2026-01-
04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHN | 2026-01-05T00:00:00.000000Z - 2026-01-
05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHN | 2026-01-06T00:00:00.000000Z - 2026-01-
06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHN | 2026-01-07T00:00:00.000000Z - 2026-01-
07T12:08:56.110000Z | 100.0 Hz, 4373612 samples
2Z.GE2..HHZ | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHZ | 2026-01-02T00:00:00.000000Z - 2026-01-
02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHZ | 2026-01-03T00:00:00.000000Z - 2026-01-
03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHZ | 2026-01-04T00:00:00.000000Z - 2026-01-
04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHZ | 2026-01-05T00:00:00.000000Z - 2026-01-
05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHZ | 2026-01-06T00:00:00.000000Z - 2026-01-
06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHZ | 2026-01-07T00:00:00.000000Z - 2026-01-
07T12:08:56.110000Z | 100.0 Hz, 4373612 samples
2Z.GE4..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHE | 2026-01-02T00:00:00.000000Z - 2026-01-
02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHE | 2026-01-03T00:00:00.000000Z - 2026-01-
03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHE | 2026-01-04T00:00:00.000000Z - 2026-01-
04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
```

2Z.GE4..HHE | 2026-01-05T00:00:00.000000Z - 2026-01-05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHE | 2026-01-06T00:00:00.000000Z - 2026-01-06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHE | 2026-01-07T00:00:00.000000Z - 2026-01-07T11:15:25.340000Z | 100.0 Hz, 4052535 samples
2Z.GE4..HHN | 2026-01-01T00:00:00.000000Z - 2026-01-01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHN | 2026-01-02T00:00:00.000000Z - 2026-01-02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHN | 2026-01-03T00:00:00.000000Z - 2026-01-03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHN | 2026-01-04T00:00:00.000000Z - 2026-01-04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHN | 2026-01-05T00:00:00.000000Z - 2026-01-05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHN | 2026-01-06T00:00:00.000000Z - 2026-01-06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHN | 2026-01-07T00:00:00.000000Z - 2026-01-07T11:15:25.340000Z | 100.0 Hz, 4052535 samples
2Z.GE4..HHZ | 2026-01-01T00:00:00.000000Z - 2026-01-01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHZ | 2026-01-02T00:00:00.000000Z - 2026-01-02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHZ | 2026-01-03T00:00:00.000000Z - 2026-01-03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHZ | 2026-01-04T00:00:00.000000Z - 2026-01-04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHZ | 2026-01-05T00:00:00.000000Z - 2026-01-05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHZ | 2026-01-06T00:00:00.000000Z - 2026-01-06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE4..HHZ | 2026-01-07T00:00:00.000000Z - 2026-01-07T11:15:25.340000Z | 100.0 Hz, 4052535 samples
2Z.GE5..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHE | 2026-01-02T00:00:00.000000Z - 2026-01-02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHE | 2026-01-03T00:00:00.000000Z - 2026-01-03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHE | 2026-01-04T00:00:00.000000Z - 2026-01-04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHE | 2026-01-05T00:00:00.000000Z - 2026-01-05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHE | 2026-01-06T00:00:00.000000Z - 2026-01-06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHE | 2026-01-07T00:00:00.000000Z - 2026-01-07T13:00:20.870000Z | 100.0 Hz, 4682088 samples
2Z.GE5..HHN | 2026-01-01T00:00:00.000000Z - 2026-01-01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHN | 2026-01-02T00:00:00.000000Z - 2026-01-02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHN | 2026-01-03T00:00:00.000000Z - 2026-01-03T23:59:59.990000Z | 100.0 Hz, 8640000 samples

```

03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHN | 2026-01-04T00:00:00.000000Z - 2026-01-
04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHN | 2026-01-05T00:00:00.000000Z - 2026-01-
05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHN | 2026-01-06T00:00:00.000000Z - 2026-01-
06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHN | 2026-01-07T00:00:00.000000Z - 2026-01-
07T13:00:20.870000Z | 100.0 Hz, 4682088 samples
2Z.GE5..HHZ | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHZ | 2026-01-02T00:00:00.000000Z - 2026-01-
02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHZ | 2026-01-03T00:00:00.000000Z - 2026-01-
03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHZ | 2026-01-04T00:00:00.000000Z - 2026-01-
04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHZ | 2026-01-05T00:00:00.000000Z - 2026-01-
05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHZ | 2026-01-06T00:00:00.000000Z - 2026-01-
06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE5..HHZ | 2026-01-07T00:00:00.000000Z - 2026-01-
07T13:00:20.870000Z | 100.0 Hz, 4682088 samples

```

```

In [6]: tr = stream[0]

print(tr)
print(tr.stats)

```

```

2Z.GE2..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
    network: 2Z
    station: GE2
    location:
    channel: HHE
    starttime: 2026-01-01T00:00:00.000000Z
    endtime: 2026-01-01T23:59:59.990000Z
    sampling_rate: 100.0
    delta: 0.01
    npts: 8640000
    calib: 1.0
    _format: MSEED
    mseed: AttribDict({'dataquality': 'D',
'number_of_records': 4541, 'encoding': 'STEIM2', 'byteorder': '>',
'record_length': 4096, 'filesize': 1048576})

```

```
In [7]: stations = sorted(set(tr.stats.station for tr in stream))
channels = sorted(set(tr.stats.channel for tr in stream))
sampling_rates = sorted(set(tr.stats.sampling_rate for tr in stream))

print("Stations:", stations)
print("Channels:", channels)
print("Sampling rates:", sampling_rates)
```

```
Stations: ['GE2', 'GE4', 'GE5']
Channels: ['HHE', 'HHN', 'HHZ']
Sampling rates: [100.0]
```

```
In [8]: from collections import Counter

station_count = Counter(tr.stats.station for tr in stream)
channel_count = Counter(tr.stats.channel for tr in stream)

print("Traces per station:", station_count)
print("Traces per channel:", channel_count)
```

```
Traces per station: Counter({'GE2': 21, 'GE4': 21, 'GE5': 21})
Traces per channel: Counter({'HHE': 21, 'HHN': 21, 'HHZ': 21})
```

```
In [9]: start_times = [tr.stats.starttime for tr in stream]
end_times = [tr.stats.endtime for tr in stream]

print("Dataset start:", min(start_times))
print("Dataset end:", max(end_times))
```

```
Dataset start: 2026-01-01T00:00:00.000000Z
Dataset end: 2026-01-07T13:00:20.870000Z
```

```
In [10]: print(tr.stats)
```

```
network: 2Z
station: GE2
location:
channel: HHE
starttime: 2026-01-01T00:00:00.000000Z
endtime: 2026-01-01T23:59:59.990000Z
sampling_rate: 100.0
delta: 0.01
npts: 8640000
calib: 1.0
_format: MSEED
mseed: AttribDict({'dataquality': 'D',
'number_of_records': 4541, 'encoding': 'STEIM2', 'byteorder': '>',
'record_length': 4096, 'filesize': 1048576})
```

4. Visual Inspection of the Raw Data

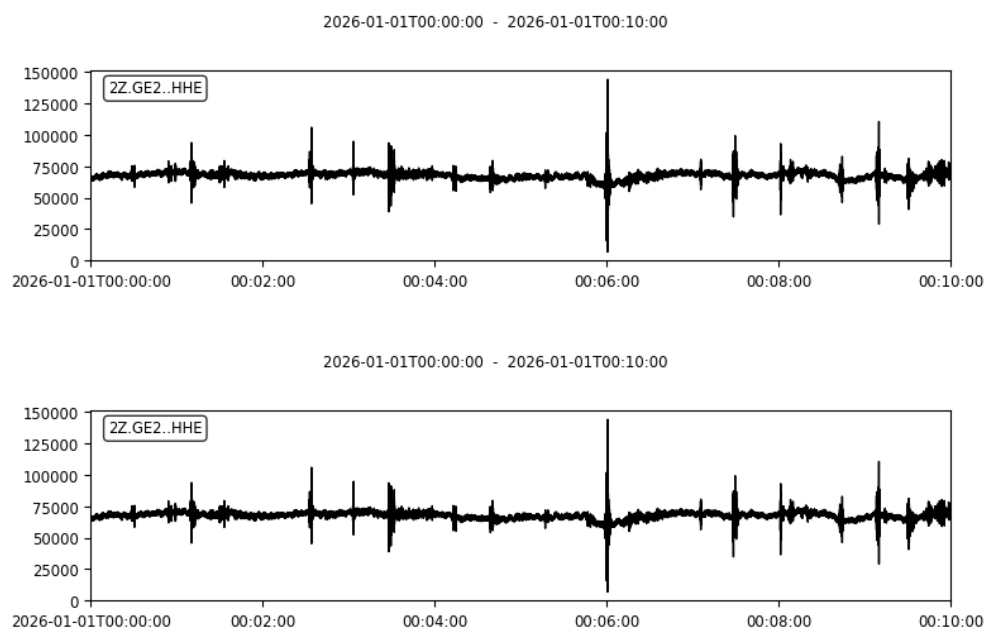
The continuous recordings are visually inspected to identify possible geyser eruptions before applying any preprocessing.

```
In [11]: tr = stream[0]
```

```
In [12]: from obspy import UTCDateTime

start = tr.stats.starttime
end = start + 10 * 60    # 10 minutes

tr.slice(start, end).plot()
```



Short Raw-Data Inspection

A short raw waveform window around a clear transient is inspected before preprocessing. The event is displayed in the time and frequency domains to support the later filter selection.

```
In [6]: from obspy import UTCDateTime

inspection_time = UTCDateTime(
    "2026-01-02T09:50:23.670000Z"
)

raw_inspection = stream.select(
    station="GE2",
    channel="HHE"
).copy()

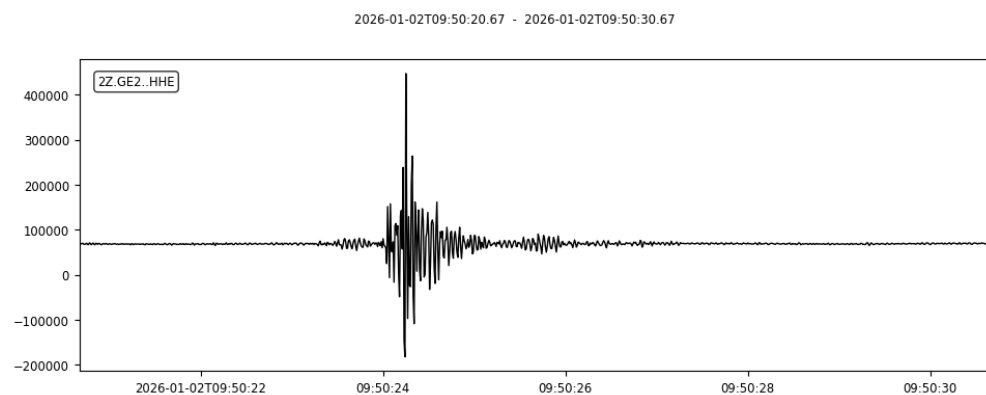
raw_inspection.trim(
    starttime=inspection_time - 3,
    endtime=inspection_time + 7
)

print(raw_inspection)
```

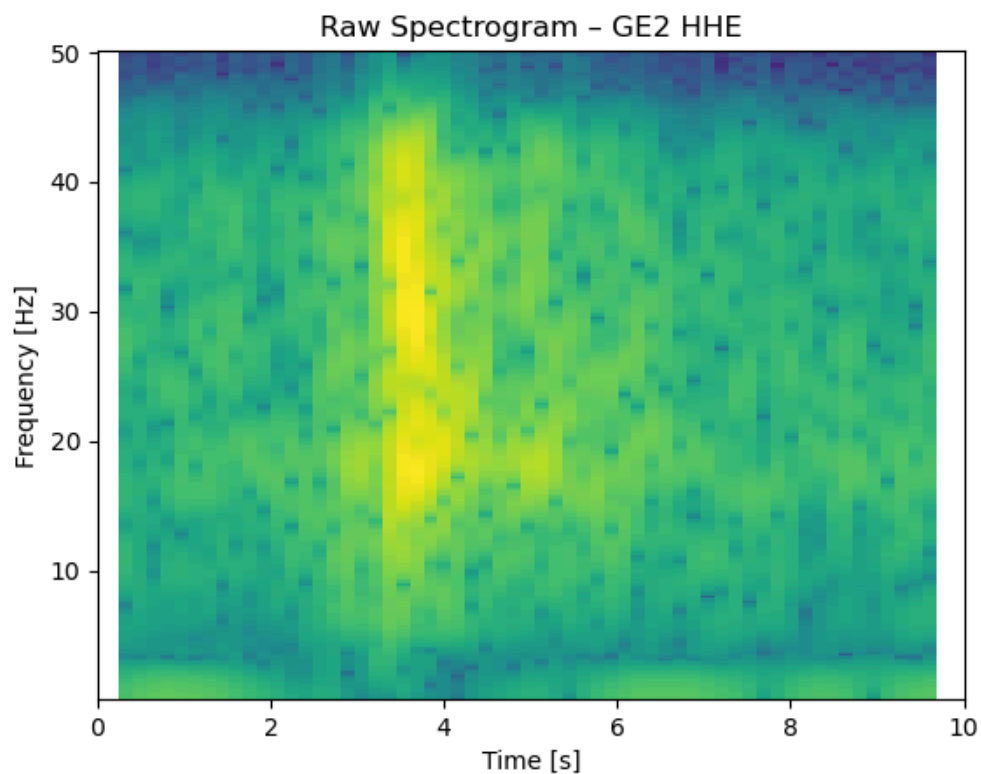
1 Trace(s) in Stream:
 2Z.GE2..HHE | 2026-01-02T09:50:20.670000Z - 2026-01-02T09:50:30.670000Z | 100.0 Hz, 1001 samples

```
In [7]: fig = raw_inspection.plot(
    size=(1000, 400),
    equal_scale=False,
    title="Raw Waveform - GE2 HHE"
)

plt.show(fig)
```



```
In [8]: raw_inspection.spectrogram(  
        log=False,  
        wlen=0.5,  
        per_lap=0.75,  
        dbscale=True,  
        title="Raw Spectrogram - GE2 HHE"  
    )
```



[None]

5. Data Preprocessing

Before event detection, a copy of the raw trace is preprocessed so that the original recording remains unchanged. The processing sequence follows the course workflow: demeaning, linear detrending, tapering, and band-pass filtering.

```
In [9]: tr = stream.select(
        station="GE2",
        channel="HHE"
    )[0]

    tr_proc = tr.copy()

    print("Mean before demeaning:", tr_proc.data.mean())

    tr_proc.detrend("demean")

    print("Mean after demeaning:", tr_proc.data.mean())

    tr_proc.detrend("linear")

    tr_proc.taper(
        type="hann",
        max_percentage=0.05
    )
```

```
Mean before demeaning: 67165.03427592592
Mean after demeaning: 3.973642985026042e-12
```

```
Out[9]: 2Z.GE2..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
        01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
```

```
In [11]: raw_example = raw_inspection[0].copy()
processed_example = raw_example.copy()

processed_example.detrend("demean")
processed_example.detrend("linear")

processed_example.taper(
    type="hann",
    max_percentage=0.05
)

raw_time = raw_example.times(
    reftime=inspection_time
)

processed_time = processed_example.times(
    reftime=inspection_time
)

fig, axes = plt.subplots(
    2,
    1,
    figsize=(12, 7),
    sharex=True
)

axes[0].plot(
    raw_time,
    raw_example.data,
    linewidth=0.7
)

axes[0].set_title("Raw Waveform")
axes[0].set_ylabel("Amplitude [counts]")
axes[0].grid()

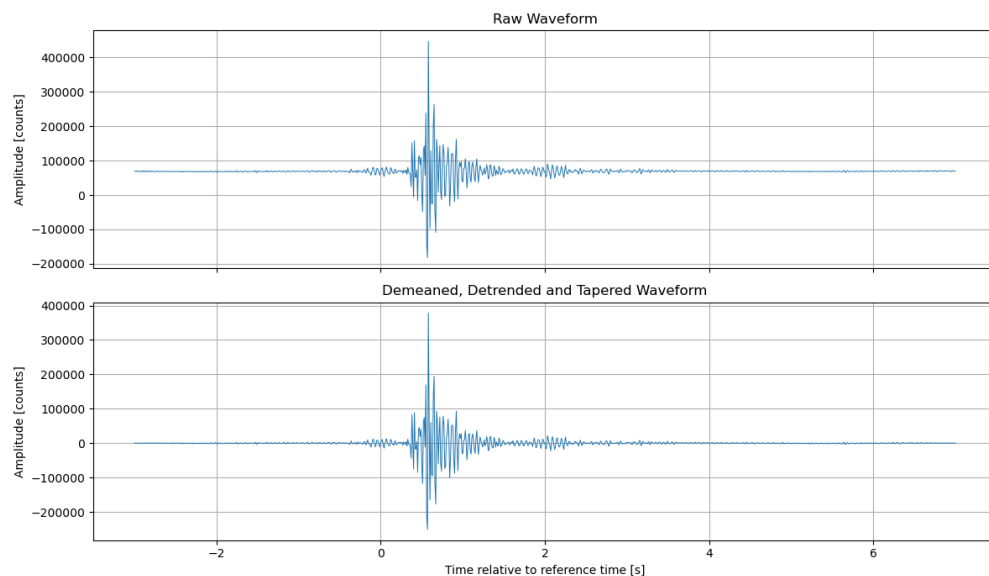
axes[1].plot(
    processed_time,
    processed_example.data,
    linewidth=0.7
)

axes[1].set_title(
    "Demeaned, Detrended and Tapered Waveform"
)

axes[1].set_xlabel(
    "Time relative to reference time [s]"
)

axes[1].set_ylabel("Amplitude [counts]")
axes[1].grid()
```

```
plt.tight_layout()  
plt.show()
```



The preprocessing removes the mean offset and linear trend while preserving the transient waveform. A Hann taper is applied to reduce edge effects before spectral analysis and filtering.

Preprocessing of the Candidate-Event Window

Before detecting geyser eruptions, the seismic data are preprocessed to remove trends and prepare the signal for further analysis.

To preserve the original recordings, all preprocessing is performed on a copy of the selected trace.

```
In [30]: tr_proc = tr.copy()

# 1. Remove the mean
tr_proc.detrend("demean")

# 2. Remove any linear trend
tr_proc.detrend("linear")

# 3. Apply a taper
tr_proc.taper(type="hann", max_percentage=0.05)

# 4. Filter (we'll choose the correct filter next)
```

```
Out[30]: 2Z.GE2..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
```

```
In [12]: event = stream.select(
            station="GE2",
            channel="HHE"
        ).copy()

event.trim(
    starttime=inspection_time - 3,
    endtime=inspection_time + 7
)

event = event[0]

print(event)
```

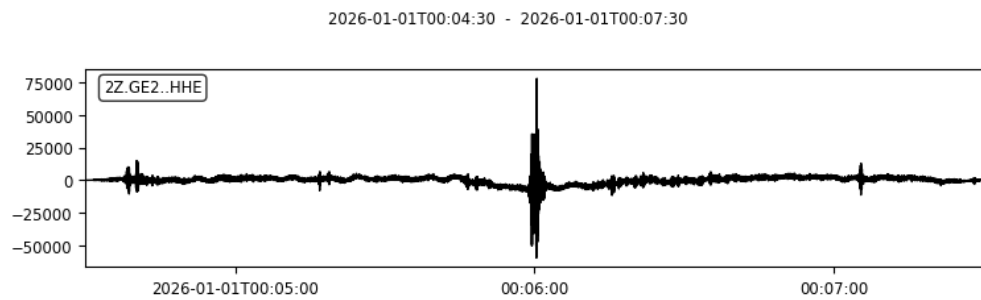
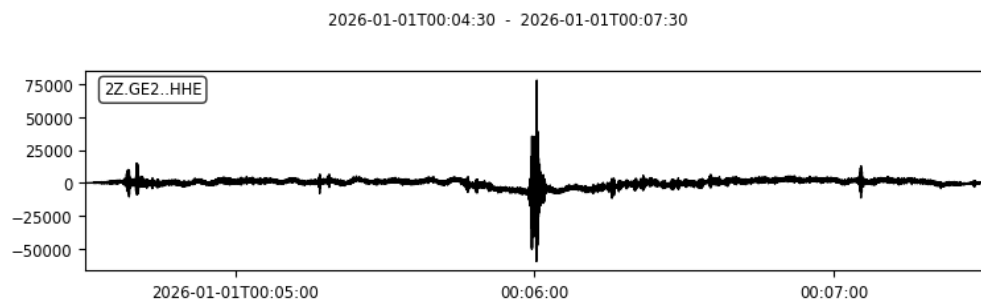
```
2Z.GE2..HHE | 2026-01-02T09:50:20.670000Z - 2026-01-
02T09:50:30.670000Z | 100.0 Hz, 1001 samples
```

```
In [13]: event.detrend("demean")
event.detrend("linear")

event.taper(
    type="hann",
    max_percentage=0.05
)
```

```
Out[13]: 2Z.GE2..HHE | 2026-01-02T09:50:20.670000Z - 2026-01-
02T09:50:30.670000Z | 100.0 Hz, 1001 samples
```

```
In [34]: event.plot()
```



```
In [14]: NFFT = 256
noverlap = 128

event_time = event.times(
    reftime=inspection_time
)

fig = plt.figure(figsize=(14, 8))

ax1 = fig.add_subplot(2, 1, 1)

ax1.plot(
    event_time,
    event.data,
    linewidth=0.5
)

ax1.axvline(
    0,
    linestyle="--",
    label="Reference time"
)

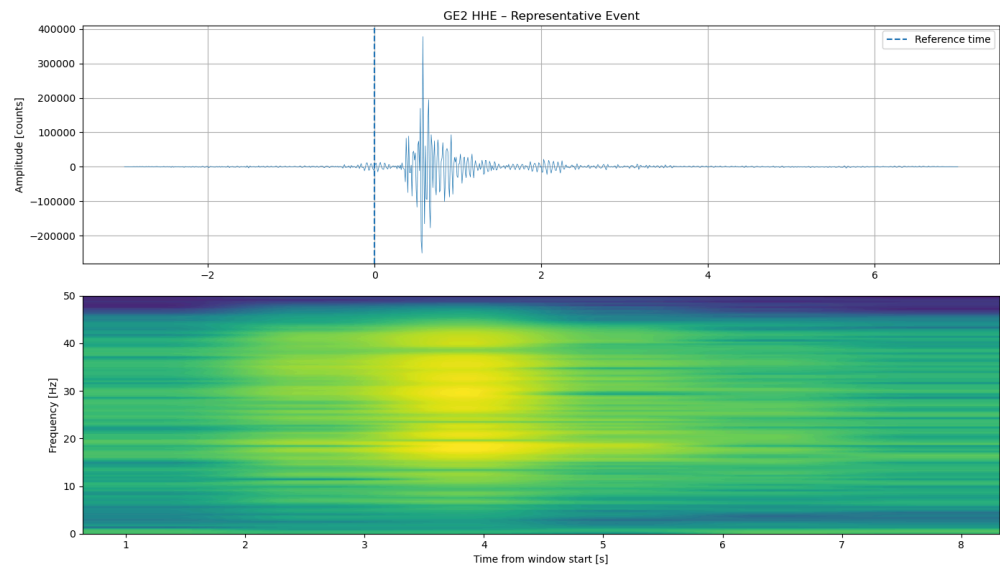
ax1.set_ylabel("Amplitude [counts]")
ax1.set_title("GE2 HHE - Representative Event")
ax1.legend()
ax1.grid()

ax2 = fig.add_subplot(2, 1, 2)

Pxx, frequencies, bins, image = ax2.specgram(
    event.data,
    NFFT=NFFT,
    Fs=event.stats.sampling_rate,
    noverlap=noverlap
)

ax2.set_xlabel("Time from window start [s]")
ax2.set_ylabel("Frequency [Hz]")
ax2.set_ylim(0, 50)

plt.tight_layout()
plt.show()
```



Frequency Spectrum of the Candidate Event

A Fourier transform is applied to the representative event to examine its overall frequency content. Unlike the spectrogram, which shows how frequency content changes through time, the frequency spectrum shows the total amplitude associated with each frequency.

```

In [9]: # Sampling information
sampling_rate = event.stats.sampling_rate
dt = event.stats.delta
nyquist = sampling_rate / 2

# Fourier transform
event_fft = np.fft.rfft(event.data)

# Frequency axis
frequencies = np.linspace(
    0,
    nyquist,
    len(event_fft)
)

# Normalized amplitude spectrum
amplitude_spectrum = (
    2.0 / len(event.data)
    * np.abs(event_fft)
)

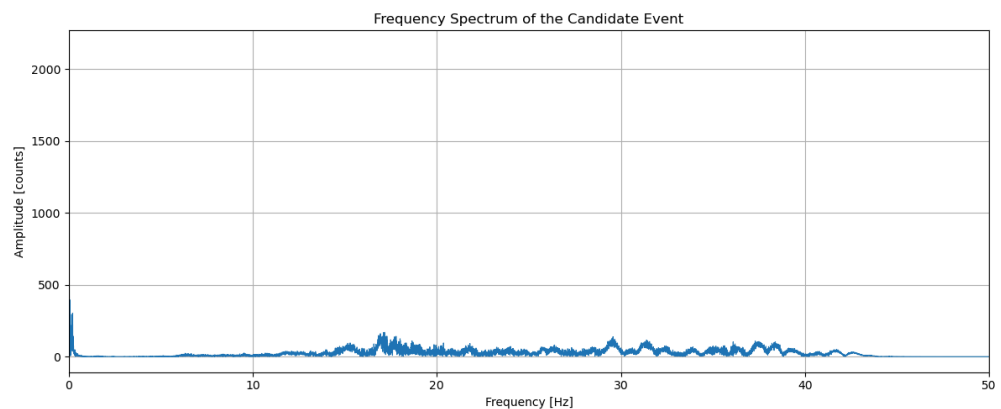
# Plot
plt.figure(figsize=(12, 5))

plt.plot(
    frequencies,
    amplitude_spectrum,
    linewidth=0.8
)

plt.xlabel("Frequency [Hz]")
plt.ylabel("Amplitude [counts]")
plt.title("Frequency Spectrum of the Candidate Event")
plt.xlim(0, 50)
plt.grid()

plt.tight_layout()
plt.show()

```



```
In [16]: event = stream.select(
            station="GE2",
            channel="HHE"
        ).copy()

        event.trim(
            starttime=inspection_time - 3,
            endtime=inspection_time + 7
        )

        event = event[0]

        event.detrend("demean")
        event.detrend("linear")

        event.taper(
            type="hann",
            max_percentage=0.05
        )

        print(event)
```

2Z.GE2..HHE | 2026-01-02T09:50:20.670000Z - 2026-01-02T09:50:30.670000Z | 100.0 Hz, 1001 samples

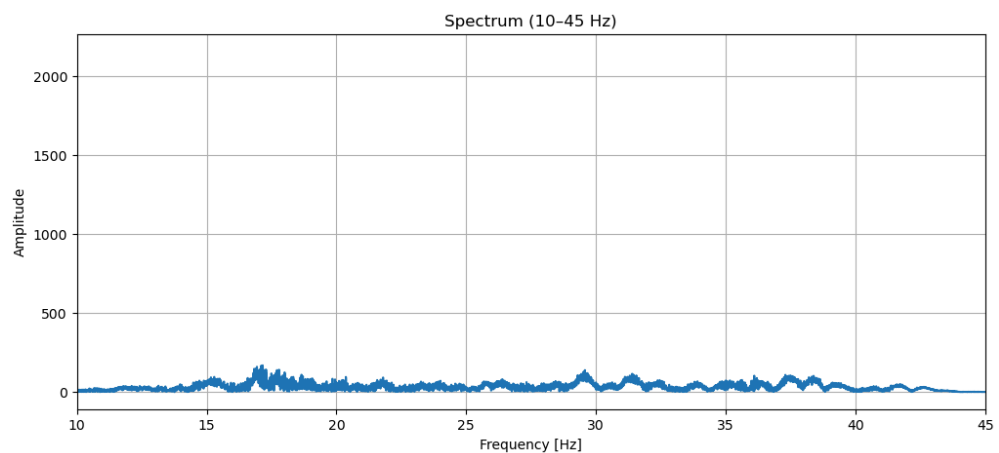
```
In [11]: plt.figure(figsize=(12,5))

        plt.plot(frequencies, amplitude_spectrum)

        plt.xlim(10, 45)
        plt.grid()

        plt.xlabel("Frequency [Hz]")
        plt.ylabel("Amplitude")
        plt.title("Spectrum (10-45 Hz)")

        plt.show()
```



```
In [13]: freqmin = 10  
         freqmax = 40
```

```
In [14]: event_filtered = event.copy()  
  
         event_filtered.filter(  
             "bandpass",  
             freqmin=10,  
             freqmax=40,  
             corners=4,  
             zerophase=True  
         )  
  
         print(event_filtered)
```

```
2Z.GE2..HHE | 2026-01-01T00:04:30.000000Z - 2026-01-  
01T00:07:30.000000Z | 100.0 Hz, 18001 samples
```

```

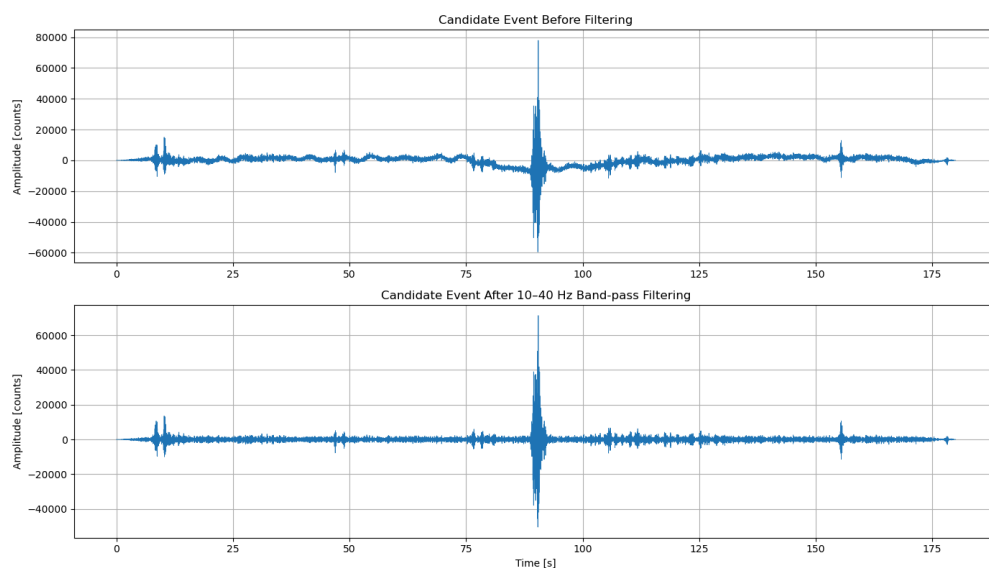
In [15]: fig = plt.figure(figsize=(14, 8))

ax1 = fig.add_subplot(2, 1, 1)
ax1.plot(
    event.times(),
    event.data,
    linewidth=0.6
)
ax1.set_ylabel("Amplitude [counts]")
ax1.set_title("Candidate Event Before Filtering")
ax1.grid()

ax2 = fig.add_subplot(2, 1, 2, sharex=ax1)
ax2.plot(
    event_filtered.times(),
    event_filtered.data,
    linewidth=0.6
)
ax2.set_xlabel("Time [s]")
ax2.set_ylabel("Amplitude [counts]")
ax2.set_title("Candidate Event After 10-40 Hz Band-pass Filtering")
ax2.grid()

plt.tight_layout()
plt.show()

```



```

In [16]: fig = plt.figure(figsize=(14, 8))

ax1 = fig.add_subplot(2, 1, 1)
ax1.plot(
    event_filtered.times(),
    event_filtered.data,
    linewidth=0.5
)
ax1.set_ylabel("Amplitude [counts]")
ax1.set_title("Filtered Candidate Event")
ax1.grid()

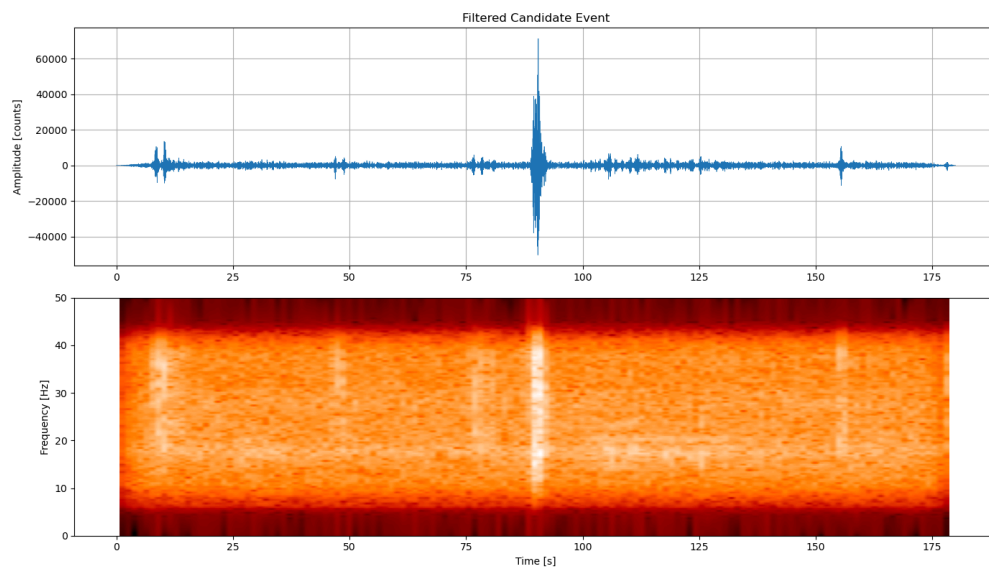
ax2 = fig.add_subplot(2, 1, 2, sharex=ax1)

ax2.specgram(
    event_filtered.data,
    NFFT=256,
    Fs=event_filtered.stats.sampling_rate,
    noverlap=128,
    cmap=plt.cm.gist_heat
)

ax2.set_xlabel("Time [s]")
ax2.set_ylabel("Frequency [Hz]")
ax2.set_ylim(0, 50)

plt.tight_layout()
plt.show()

```



Event Detection in the Complete Dataset

The GE2 HHE component is used as the reference channel for detecting candidate eruption events over the complete seven-day recording period. Detected events are later verified using the remaining stations and components.

```
In [18]: ge2_hhe = stream.select(
          station="GE2",
          channel="HHE"
        ).copy()

ge2_hhe.sort()

print(ge2_hhe)
```

```
7 Trace(s) in Stream:
2Z.GE2..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-02T00:00:00.000000Z - 2026-01-
02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-03T00:00:00.000000Z - 2026-01-
03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-04T00:00:00.000000Z - 2026-01-
04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-05T00:00:00.000000Z - 2026-01-
05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-06T00:00:00.000000Z - 2026-01-
06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-07T00:00:00.000000Z - 2026-01-
07T12:08:56.110000Z | 100.0 Hz, 4373612 samples
```

```
In [19]: ge2_hhe_processed = ge2_hhe.copy()

for trace in ge2_hhe_processed:

    trace.detrend("demean")
    trace.detrend("linear")

    trace.taper(
        type="hann",
        max_percentage=0.05,
        max_length=10
    )

    trace.filter(
        "bandpass",
        freqmin=10,
        freqmax=40,
        corners=4,
        zerophase=True
    )

print(ge2_hhe_processed)
```

```
7 Trace(s) in Stream:
2Z.GE2..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-02T00:00:00.000000Z - 2026-01-
02T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-03T00:00:00.000000Z - 2026-01-
03T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-04T00:00:00.000000Z - 2026-01-
04T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-05T00:00:00.000000Z - 2026-01-
05T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-06T00:00:00.000000Z - 2026-01-
06T23:59:59.990000Z | 100.0 Hz, 8640000 samples
2Z.GE2..HHE | 2026-01-07T00:00:00.000000Z - 2026-01-
07T12:08:56.110000Z | 100.0 Hz, 4373612 samples
```

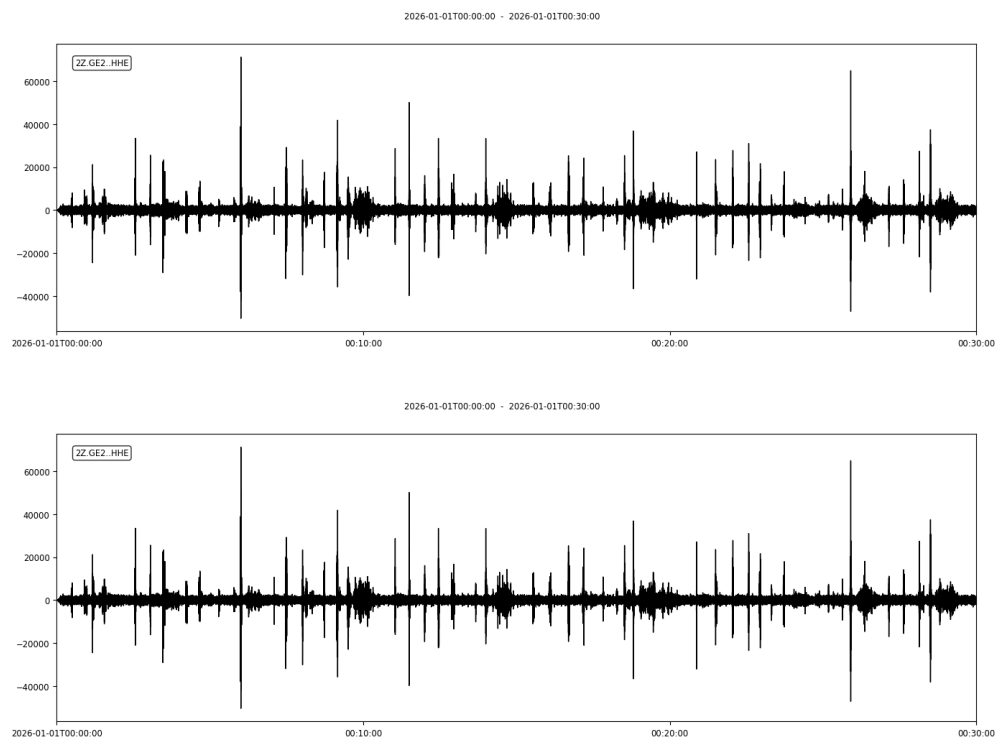
```
In [20]: first_day = ge2_hhe_processed[0]

inspection_start = first_day.stats.starttime
inspection_end = inspection_start + 30 * 60

first_30_minutes = first_day.copy()

first_30_minutes.trim(
    starttime=inspection_start,
    endtime=inspection_end
)

first_30_minutes.plot(
    size=(1400, 500),
    title="GE2 HHE - First 30 Minutes After 10-40 Hz Filtering"
)
```



Initial STA/LTA Parameter Exploration

This histogram shows the distribution of STA/LTA values for the initial test using a 0.5 s short-term window and a 10 s long-term window. It was used as an exploratory step to examine the characteristic-function range. The final event detection uses longer windows of STA = 2 s and LTA = 20 s, with trigger thresholds of 4 and 1.5.

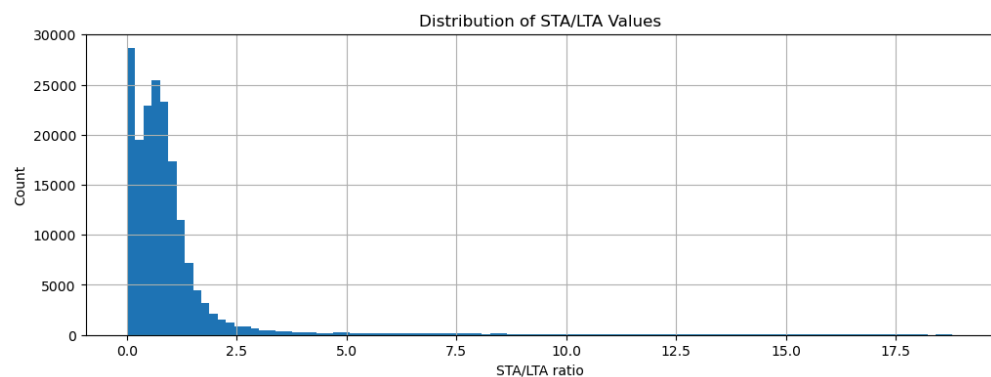
```
In [25]: plt.figure(figsize=(12,4))

plt.hist(
    characteristic_function,
    bins=100
)

plt.xlabel("STA/LTA ratio")
plt.ylabel("Count")
plt.title("Distribution of STA/LTA Values")

plt.grid()

plt.show()
```



```
In [26]: trigger_on = 6
trigger_off = 2

triggers = trigger_onset(
    characteristic_function,
    trigger_on,
    trigger_off
)

print(f"Detected events: {len(triggers)}")
```

Detected events: 76

```
In [27]: for i, (on, off) in enumerate(triggers, start=1):
        onset = first_30_minutes.stats.starttime + on / sampling_rate
        duration = (off - on) / sampling_rate

        print(
            f"{i:2d} | "
            f"{onset} | "
            f"{duration:.2f} s"
        )
```

```
1 | 2026-01-01T00:00:28.780000Z | 0.64 s
2 | 2026-01-01T00:00:30.570000Z | 0.52 s
3 | 2026-01-01T00:00:54.140000Z | 0.55 s
4 | 2026-01-01T00:00:56.350000Z | 0.43 s
5 | 2026-01-01T00:01:09.370000Z | 1.66 s
6 | 2026-01-01T00:01:30.020000Z | 0.59 s
7 | 2026-01-01T00:01:33.110000Z | 0.45 s
8 | 2026-01-01T00:02:32.380000Z | 0.92 s
9 | 2026-01-01T00:02:33.850000Z | 0.71 s
10 | 2026-01-01T00:03:02.840000Z | 1.02 s
11 | 2026-01-01T00:03:27.580000Z | 0.87 s
12 | 2026-01-01T00:03:29.290000Z | 0.55 s
13 | 2026-01-01T00:04:12.830000Z | 0.62 s
14 | 2026-01-01T00:04:14.720000Z | 0.58 s
15 | 2026-01-01T00:04:38.290000Z | 0.97 s
16 | 2026-01-01T00:04:40.400000Z | 0.57 s
17 | 2026-01-01T00:05:16.970000Z | 0.53 s
18 | 2026-01-01T00:05:46.590000Z | 0.64 s
19 | 2026-01-01T00:05:59.100000Z | 2.11 s
20 | 2026-01-01T00:06:15.810000Z | 0.63 s
21 | 2026-01-01T00:07:05.340000Z | 0.82 s
22 | 2026-01-01T00:07:27.830000Z | 1.03 s
23 | 2026-01-01T00:07:29.390000Z | 0.52 s
24 | 2026-01-01T00:08:00.920000Z | 1.11 s
25 | 2026-01-01T00:08:42.430000Z | 0.79 s
26 | 2026-01-01T00:08:44.090000Z | 0.69 s
27 | 2026-01-01T00:09:07.930000Z | 1.08 s
28 | 2026-01-01T00:09:09.590000Z | 0.75 s
29 | 2026-01-01T00:09:29.520000Z | 2.40 s
30 | 2026-01-01T00:09:44.250000Z | 0.92 s
31 | 2026-01-01T00:11:02.220000Z | 0.69 s
32 | 2026-01-01T00:11:29.270000Z | 0.99 s
33 | 2026-01-01T00:11:59.870000Z | 1.15 s
34 | 2026-01-01T00:12:26.660000Z | 0.93 s
35 | 2026-01-01T00:12:52.760000Z | 0.67 s
36 | 2026-01-01T00:12:54.420000Z | 0.66 s
37 | 2026-01-01T00:12:56.950000Z | 0.63 s
38 | 2026-01-01T00:13:40.000000Z | 0.72 s
39 | 2026-01-01T00:13:58.780000Z | 1.96 s
40 | 2026-01-01T00:14:23.390000Z | 0.75 s
41 | 2026-01-01T00:14:26.820000Z | 0.45 s
42 | 2026-01-01T00:15:31.650000Z | 0.70 s
43 | 2026-01-01T00:15:32.890000Z | 0.66 s
```

44		2026-01-01T00:16:03.580000Z		0.82 s
45		2026-01-01T00:16:05.080000Z		0.71 s
46		2026-01-01T00:16:41.270000Z		0.99 s
47		2026-01-01T00:17:09.540000Z		1.07 s
48		2026-01-01T00:17:11.570000Z		0.76 s
49		2026-01-01T00:17:49.180000Z		0.76 s
50		2026-01-01T00:18:15.800000Z		0.57 s
51		2026-01-01T00:18:30.380000Z		1.90 s
52		2026-01-01T00:18:48.370000Z		0.96 s
53		2026-01-01T00:19:02.230000Z		0.57 s
54		2026-01-01T00:19:03.610000Z		0.63 s
55		2026-01-01T00:19:27.540000Z		0.69 s
56		2026-01-01T00:20:52.010000Z		0.91 s
57		2026-01-01T00:21:29.380000Z		0.85 s
58		2026-01-01T00:22:02.280000Z		1.84 s
59		2026-01-01T00:22:34.150000Z		0.98 s
60		2026-01-01T00:22:55.300000Z		1.22 s
61		2026-01-01T00:22:56.920000Z		0.84 s
62		2026-01-01T00:23:18.950000Z		0.76 s
63		2026-01-01T00:23:42.030000Z		0.86 s
64		2026-01-01T00:23:43.940000Z		0.67 s
65		2026-01-01T00:24:24.690000Z		0.77 s
66		2026-01-01T00:25:10.520000Z		0.58 s
67		2026-01-01T00:25:37.680000Z		0.52 s
68		2026-01-01T00:25:52.760000Z		1.98 s
69		2026-01-01T00:26:20.200000Z		0.49 s
70		2026-01-01T00:26:21.500000Z		0.64 s
71		2026-01-01T00:27:08.500000Z		0.89 s
72		2026-01-01T00:27:37.120000Z		0.90 s
73		2026-01-01T00:27:38.600000Z		0.53 s
74		2026-01-01T00:28:07.730000Z		1.00 s
75		2026-01-01T00:28:28.650000Z		2.38 s
76		2026-01-01T00:28:48.440000Z		0.70 s

```
In [28]: plt.figure(figsize=(15,5))

plt.plot(
    first_30_minutes.times(),
    first_30_minutes.data,
    color="black",
    linewidth=0.6
)

for on, off in triggers:
    plt.axvspan(
        on / sampling_rate,
        off / sampling_rate,
        color="red",
        alpha=0.3
    )

plt.xlabel("Time [s]")
plt.ylabel("Amplitude [counts]")
plt.title("Detected Candidate Events (STA/LTA)")
plt.grid()

plt.show()
```



```

In [30]: fig = plt.figure(figsize=(14, 8))

ax1 = fig.add_subplot(2, 1, 1)

ax1.plot(
    first_30_minutes.times(),
    first_30_minutes.data,
    linewidth=0.5
)

ax1.set_ylabel("Amplitude [counts]")
ax1.set_title("Filtered Seismic Signal")
ax1.grid()

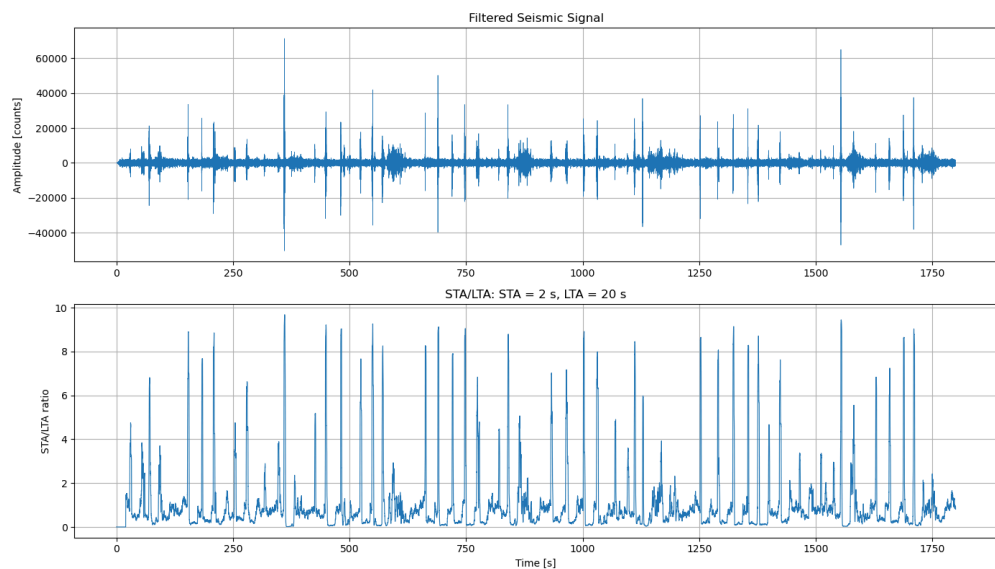
ax2 = fig.add_subplot(2, 1, 2, sharex=ax1)

ax2.plot(
    first_30_minutes.times(),
    characteristic_function_long,
    linewidth=0.8
)

ax2.set_xlabel("Time [s]")
ax2.set_ylabel("STA/LTA ratio")
ax2.set_title("STA/LTA: STA = 2 s, LTA = 20 s")
ax2.grid()

plt.tight_layout()
plt.show()

```



```
In [32]: trigger_on = 4
trigger_off = 1.5

triggers_long = trigger_onset(
    characteristic_function_long,
    trigger_on,
    trigger_off
)

print(f"Detected candidate events: {len(triggers_long)}")
```

Detected candidate events: 43

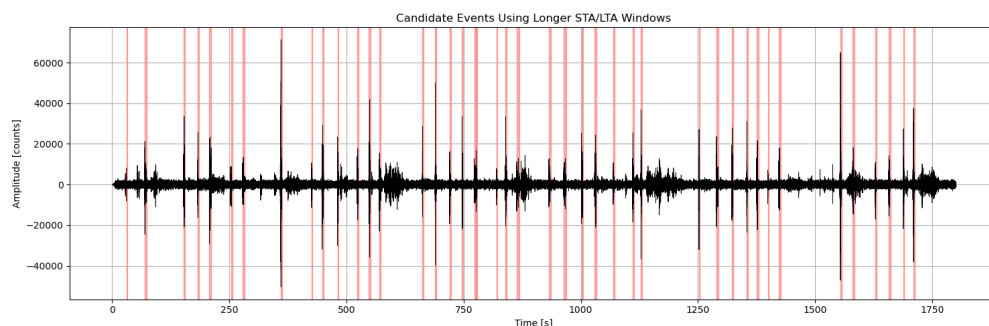
```
In [33]: plt.figure(figsize=(15, 5))

plt.plot(
    first_30_minutes.times(),
    first_30_minutes.data,
    color="black",
    linewidth=0.6
)

for on, off in triggers_long:
    plt.axvspan(
        on / sampling_rate,
        off / sampling_rate,
        color="red",
        alpha=0.3
    )

plt.xlabel("Time [s]")
plt.ylabel("Amplitude [counts]")
plt.title("Candidate Events Using Longer STA/LTA Windows")
plt.grid()

plt.tight_layout()
plt.show()
```



```
In [34]: for i, (on, off) in enumerate(triggers_long, start=1):

    onset = (
        first_30_minutes.stats.starttime
        + on / sampling_rate
    )

    duration = (off - on) / sampling_rate

    print(
        f"{i:2d} | "
        f"{onset} | "
        f"{duration:.2f} s"
    )
```

```
1 | 2026-01-01T00:00:30.480000Z | 2.06 s
2 | 2026-01-01T00:01:10.040000Z | 3.59 s
3 | 2026-01-01T00:02:32.550000Z | 3.45 s
4 | 2026-01-01T00:03:02.970000Z | 2.31 s
5 | 2026-01-01T00:03:27.630000Z | 3.70 s
6 | 2026-01-01T00:04:14.600000Z | 2.12 s
7 | 2026-01-01T00:04:38.510000Z | 3.98 s
8 | 2026-01-01T00:05:59.330000Z | 3.33 s
9 | 2026-01-01T00:07:05.490000Z | 2.15 s
10 | 2026-01-01T00:07:28.070000Z | 3.53 s
11 | 2026-01-01T00:08:00.980000Z | 2.54 s
12 | 2026-01-01T00:08:42.520000Z | 3.65 s
13 | 2026-01-01T00:09:08.030000Z | 3.72 s
14 | 2026-01-01T00:09:29.900000Z | 3.59 s
15 | 2026-01-01T00:11:02.230000Z | 2.15 s
16 | 2026-01-01T00:11:29.440000Z | 2.36 s
17 | 2026-01-01T00:11:59.980000Z | 2.56 s
18 | 2026-01-01T00:12:26.760000Z | 3.34 s
19 | 2026-01-01T00:12:52.810000Z | 3.71 s
20 | 2026-01-01T00:12:57.120000Z | 1.99 s
21 | 2026-01-01T00:13:40.220000Z | 1.99 s
22 | 2026-01-01T00:13:59.210000Z | 3.11 s
23 | 2026-01-01T00:14:23.500000Z | 5.24 s
24 | 2026-01-01T00:15:31.750000Z | 4.32 s
25 | 2026-01-01T00:16:03.710000Z | 5.20 s
26 | 2026-01-01T00:16:41.380000Z | 3.85 s
27 | 2026-01-01T00:17:09.900000Z | 3.90 s
28 | 2026-01-01T00:17:49.340000Z | 2.12 s
29 | 2026-01-01T00:18:30.620000Z | 3.25 s
30 | 2026-01-01T00:18:48.560000Z | 2.12 s
31 | 2026-01-01T00:20:52.090000Z | 2.31 s
32 | 2026-01-01T00:21:29.520000Z | 4.23 s
33 | 2026-01-01T00:22:02.360000Z | 3.22 s
34 | 2026-01-01T00:22:34.270000Z | 2.20 s
35 | 2026-01-01T00:22:55.480000Z | 3.67 s
36 | 2026-01-01T00:23:19.170000Z | 2.02 s
37 | 2026-01-01T00:23:42.140000Z | 3.93 s
38 | 2026-01-01T00:25:53.390000Z | 2.84 s
```

```

39 | 2026-01-01T00:26:20.410000Z | 3.11 s
40 | 2026-01-01T00:27:08.620000Z | 2.25 s
41 | 2026-01-01T00:27:37.360000Z | 3.24 s
42 | 2026-01-01T00:28:07.920000Z | 2.42 s
43 | 2026-01-01T00:28:29.650000Z | 2.87 s

```

```

In [35]: stations_30min = obspy.Stream()

for station in ["GE2", "GE4", "GE5"]:

    trace_station = stream.select(
        station=station,
        channel="HHE"
    )[0].copy()

    start = trace_station.stats.starttime
    end = start + 30 * 60

    trace_station.trim(
        starttime=start,
        endtime=end
    )

    trace_station.detrend("demean")
    trace_station.detrend("linear")

    trace_station.taper(
        type="hann",
        max_percentage=0.05,
        max_length=10
    )

    trace_station.filter(
        "bandpass",
        freqmin=10,
        freqmax=40,
        corners=4,
        zerophase=True
    )

    stations_30min += trace_station

print(stations_30min)

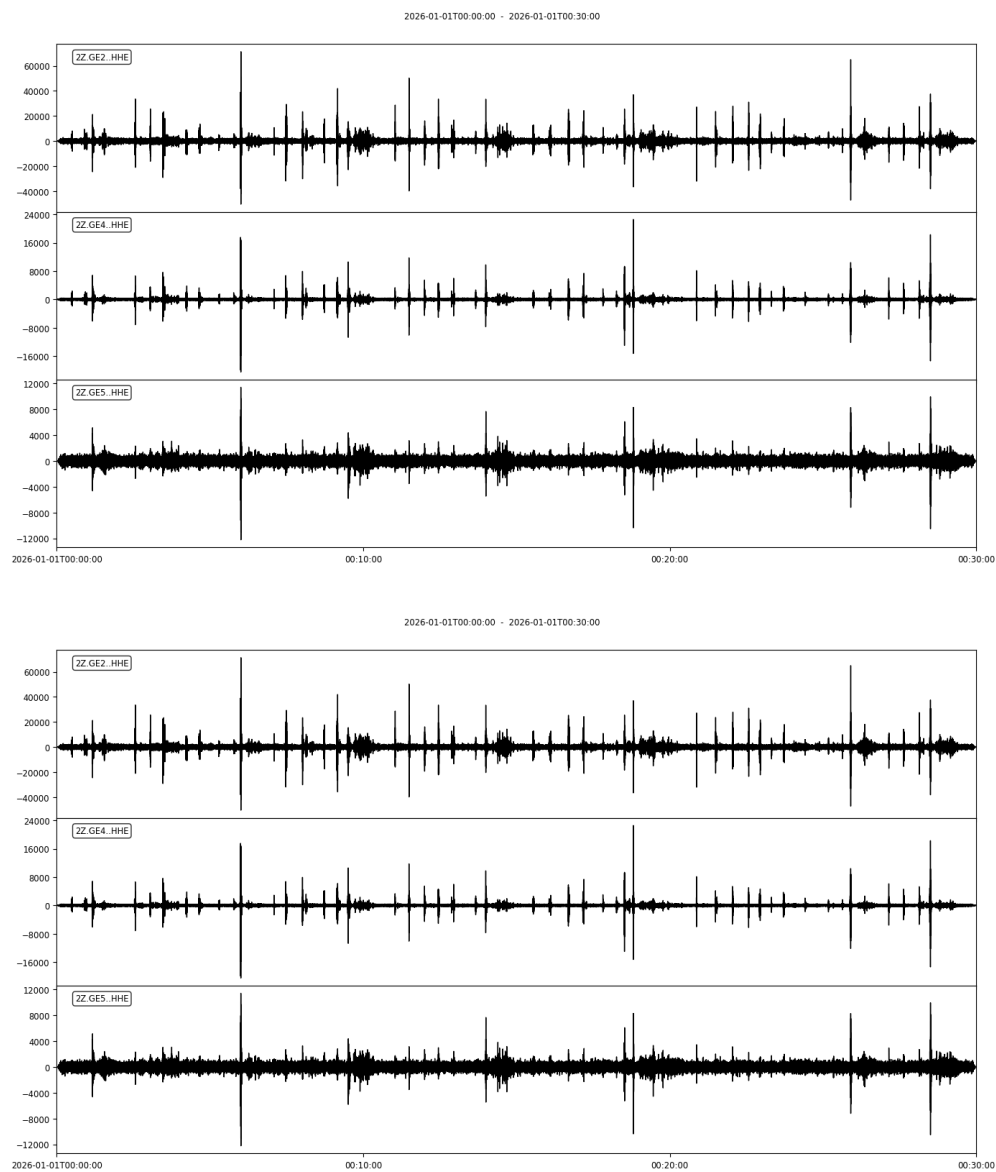
```

```

3 Trace(s) in Stream:
2Z.GE2..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T00:30:00.000000Z | 100.0 Hz, 180001 samples
2Z.GE4..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T00:30:00.000000Z | 100.0 Hz, 180001 samples
2Z.GE5..HHE | 2026-01-01T00:00:00.000000Z - 2026-01-
01T00:30:00.000000Z | 100.0 Hz, 180001 samples

```

```
In [36]: stations_30min.plot(
        size=(1400, 800),
        equal_scale=False,
        title="GE2, GE4 and GE5 - First 30 Minutes"
    )
```



```
In [37]: from obspy.signal.trigger import coincidence_trigger
```

```
In [38]: coincidence_events = coincidence_trigger(  
    trigger_type="classicstalta",  
    thr_on=4,  
    thr_off=1.5,  
    stream=stations_30min,  
    thr_coincidence_sum=2,  
    sta=2.0,  
    lta=20.0,  
    details=True  
)  
  
print(  
    f"Coincident candidate events: "  
    f"{len(coincidence_events)}"  
)
```

Coincident candidate events: 42

```
In [40]: for i, candidate in enumerate(
          coincidence_events,
          start=1
        ):
          print(
            f"{i:2d} | "
            f"{candidate['time']} | "
            f"{candidate['duration']:.2f} s | "
            f"{candidate['stations']}"
          )
```

```
1 | 2026-01-01T00:00:28.710000Z | 3.83 s | ['GE4', 'GE2']
2 | 2026-01-01T00:01:09.890000Z | 3.74 s | ['GE4', 'GE5', 'GE2']
3 | 2026-01-01T00:02:32.300000Z | 3.75 s | ['GE4', 'GE2']
4 | 2026-01-01T00:03:02.790000Z | 2.50 s | ['GE4', 'GE2']
5 | 2026-01-01T00:03:27.570000Z | 3.76 s | ['GE4', 'GE2']
6 | 2026-01-01T00:04:14.140000Z | 2.58 s | ['GE4', 'GE2']
7 | 2026-01-01T00:04:38.310000Z | 4.18 s | ['GE4', 'GE2']
8 | 2026-01-01T00:05:59.260000Z | 3.55 s | ['GE4', 'GE2', 'GE5']
9 | 2026-01-01T00:07:05.290000Z | 2.37 s | ['GE4', 'GE2']
10 | 2026-01-01T00:07:27.710000Z | 4.30 s | ['GE4', 'GE2']
11 | 2026-01-01T00:08:00.900000Z | 2.62 s | ['GE4', 'GE2']
12 | 2026-01-01T00:08:42.340000Z | 3.83 s | ['GE4', 'GE2']
13 | 2026-01-01T00:09:07.830000Z | 4.05 s | ['GE4', 'GE2']
14 | 2026-01-01T00:09:29.900000Z | 5.07 s | ['GE2', 'GE4', 'GE5']
15 | 2026-01-01T00:11:02.090000Z | 2.37 s | ['GE4', 'GE2']
16 | 2026-01-01T00:11:29.100000Z | 2.70 s | ['GE4', 'GE2']
17 | 2026-01-01T00:11:59.810000Z | 2.73 s | ['GE4', 'GE2']
18 | 2026-01-01T00:12:26.610000Z | 3.57 s | ['GE4', 'GE2']
19 | 2026-01-01T00:12:52.550000Z | 3.97 s | ['GE4', 'GE2']
20 | 2026-01-01T00:12:56.740000Z | 2.38 s | ['GE4', 'GE2']
21 | 2026-01-01T00:13:39.860000Z | 2.48 s | ['GE4', 'GE2']
22 | 2026-01-01T00:13:59.210000Z | 3.58 s | ['GE2', 'GE4', 'GE5']
23 | 2026-01-01T00:15:31.400000Z | 4.67 s | ['GE4', 'GE2']
24 | 2026-01-01T00:16:03.640000Z | 5.27 s | ['GE4', 'GE2']
25 | 2026-01-01T00:16:41.240000Z | 3.99 s | ['GE4', 'GE2']
26 | 2026-01-01T00:17:09.670000Z | 4.13 s | ['GE4', 'GE2']
27 | 2026-01-01T00:17:48.980000Z | 3.52 s | ['GE4', 'GE2']
28 | 2026-01-01T00:18:30.450000Z | 3.67 s | ['GE4', 'GE2', 'GE5']
29 | 2026-01-01T00:18:48.440000Z | 2.40 s | ['GE4', 'GE2', 'GE5']
30 | 2026-01-01T00:20:51.850000Z | 2.58 s | ['GE4', 'GE2']
31 | 2026-01-01T00:21:29.330000Z | 4.73 s | ['GE4', 'GE2']
32 | 2026-01-01T00:22:02.280000Z | 3.30 s | ['GE4', 'GE2']
33 | 2026-01-01T00:22:34.130000Z | 2.40 s | ['GE4', 'GE2']
34 | 2026-01-01T00:22:55.240000Z | 4.01 s | ['GE4', 'GE2']
35 | 2026-01-01T00:23:18.860000Z | 2.33 s | ['GE4', 'GE2']
36 | 2026-01-01T00:23:41.950000Z | 4.12 s | ['GE4', 'GE2']
37 | 2026-01-01T00:25:52.740000Z | 3.82 s | ['GE4', 'GE2', 'GE5']
38 | 2026-01-01T00:26:20.410000Z | 3.11 s | ['GE2', 'GE4']
39 | 2026-01-01T00:27:08.320000Z | 2.55 s | ['GE4', 'GE2']
40 | 2026-01-01T00:27:37.100000Z | 3.50 s | ['GE4', 'GE2']
```

```
41 | 2026-01-01T00:28:07.760000Z | 3.49 s | ['GE4', 'GE2']
42 | 2026-01-01T00:28:28.740000Z | 3.91 s | ['GE4', 'GE2', 'GE5']
```

```
In [41]: three_station_events = []

for candidate in coincidence_events:

    if len(set(candidate["stations"])) == 3:
        three_station_events.append(candidate)

print(
    f"Events recorded at all three stations: "
    f"{len(three_station_events)}"
)

for i, candidate in enumerate(
    three_station_events,
    start=1
):
    print(
        f"{i:2d} | "
        f"{candidate['time']} | "
        f"{candidate['duration']:.2f} s | "
        f"{candidate['stations']}"
    )
```

```
Events recorded at all three stations: 8
1 | 2026-01-01T00:01:09.890000Z | 3.74 s | ['GE4', 'GE5', 'GE2']
2 | 2026-01-01T00:05:59.260000Z | 3.55 s | ['GE4', 'GE2', 'GE5']
3 | 2026-01-01T00:09:29.900000Z | 5.07 s | ['GE2', 'GE4', 'GE5']
4 | 2026-01-01T00:13:59.210000Z | 3.58 s | ['GE2', 'GE4', 'GE5']
5 | 2026-01-01T00:18:30.450000Z | 3.67 s | ['GE4', 'GE2', 'GE5']
6 | 2026-01-01T00:18:48.440000Z | 2.40 s | ['GE4', 'GE2', 'GE5']
7 | 2026-01-01T00:25:52.740000Z | 3.82 s | ['GE4', 'GE2', 'GE5']
8 | 2026-01-01T00:28:28.740000Z | 3.91 s | ['GE4', 'GE2', 'GE5']
```

```
In [42]: grouping_interval = 60.0

event_groups = []

for candidate in three_station_events:

    if len(event_groups) == 0:
        event_groups.append([candidate])
        continue

    previous_time = event_groups[-1][-1]["time"]
    current_time = candidate["time"]

    time_difference = current_time - previous_time

    if time_difference <= grouping_interval:
        event_groups[-1].append(candidate)
    else:
        event_groups.append([candidate])

print(f"Probable eruption groups: {len(event_groups)}")
```

Probable eruption groups: 7

```
In [43]: for group_number, group in enumerate(
          event_groups,
          start=1
        ):

          print(f"\nGroup {group_number}")

          for candidate in group:
              print(
                  f" {candidate['time']} | "
                  f"{candidate['duration']:.2f} s"
              )
```

```
Group 1
  2026-01-01T00:01:09.890000Z | 3.74 s

Group 2
  2026-01-01T00:05:59.260000Z | 3.55 s

Group 3
  2026-01-01T00:09:29.900000Z | 5.07 s

Group 4
  2026-01-01T00:13:59.210000Z | 3.58 s

Group 5
  2026-01-01T00:18:30.450000Z | 3.67 s
  2026-01-01T00:18:48.440000Z | 2.40 s

Group 6
  2026-01-01T00:25:52.740000Z | 3.82 s

Group 7
  2026-01-01T00:28:28.740000Z | 3.91 s
```

```
In [44]: preliminary_eruptions = []

          for group in event_groups:

              first_candidate = group[0]

              preliminary_eruptions.append({
                  "time": first_candidate["time"],
                  "number_of_pulses": len(group),
                  "group_duration": (
                      group[-1]["time"]
                      + group[-1]["duration"]
                      - group[0]["time"]
                  )
              })
```

```
In [46]: print("Intervals between probable eruptions:")

for i in range(1, len(preliminary_eruptions)):

    interval = (
        preliminary_eruptions[i]["time"]
        - preliminary_eruptions[i - 1]["time"]
    )

    print(
        f"Event {i} → Event {i + 1}: "
        f"{interval / 60:.2f} minutes"
    )
```

```
Intervals between probable eruptions:
Event 1 → Event 2: 4.82 minutes
Event 2 → Event 3: 3.51 minutes
Event 3 → Event 4: 4.49 minutes
Event 4 → Event 5: 4.52 minutes
Event 5 → Event 6: 7.37 minutes
Event 6 → Event 7: 2.60 minutes
```

```
In [47]: validation_windows = []

for eruption in preliminary_eruptions:

    eruption_time = eruption["time"]

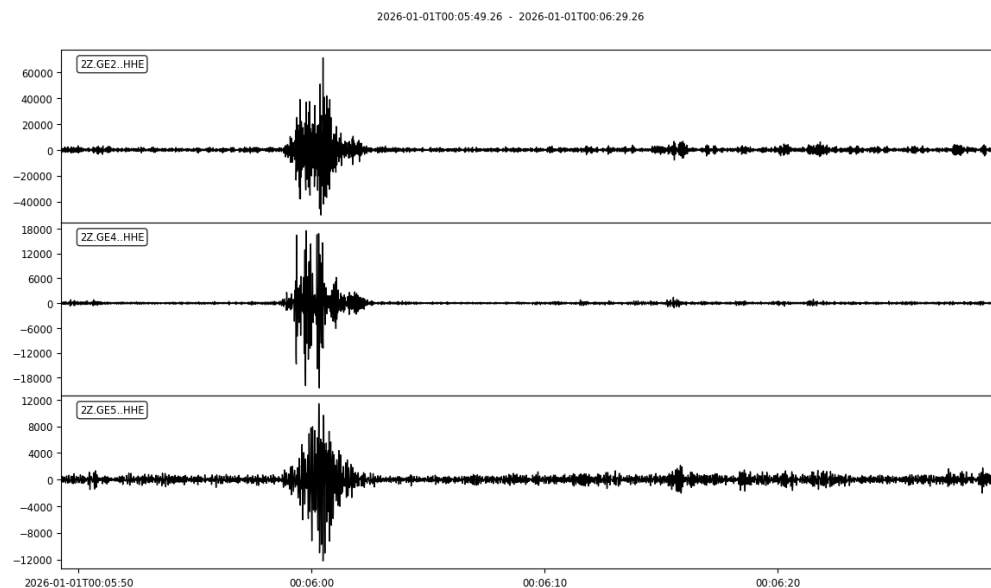
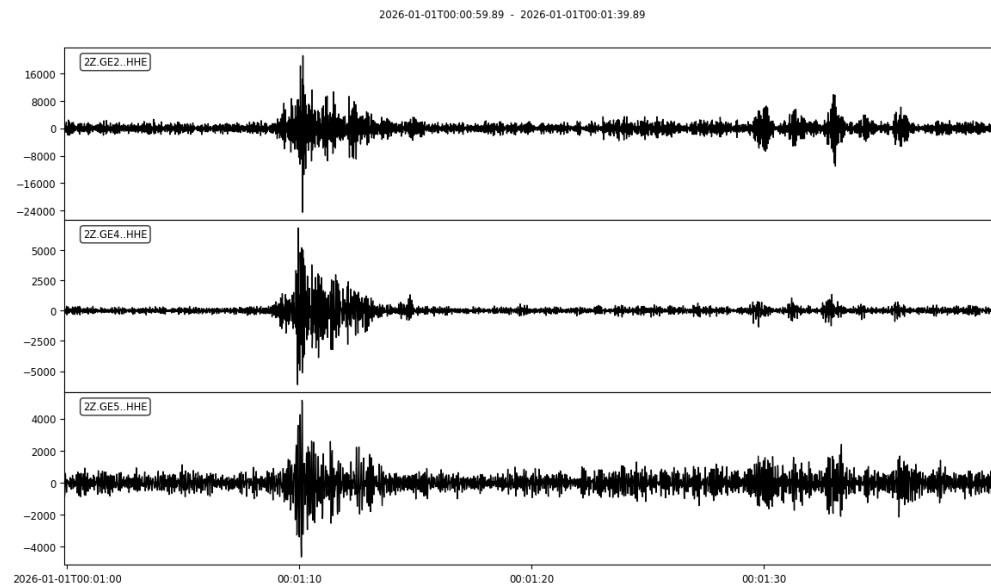
    event_window = stations_30min.copy()

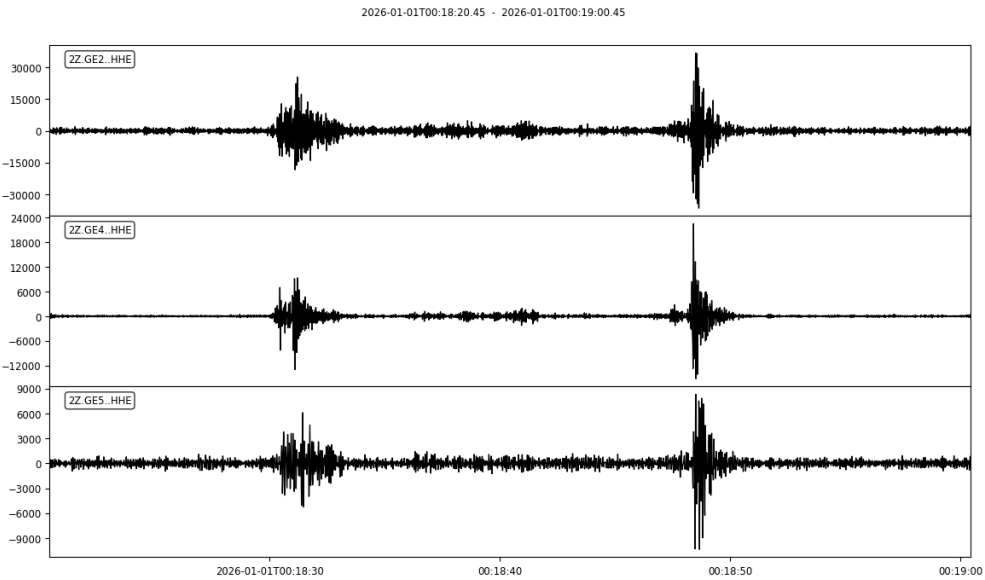
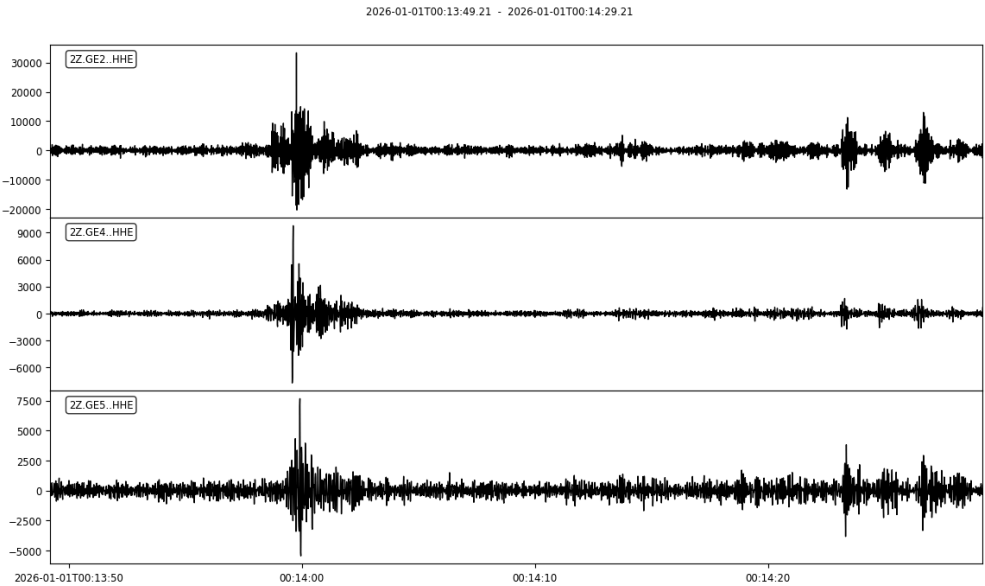
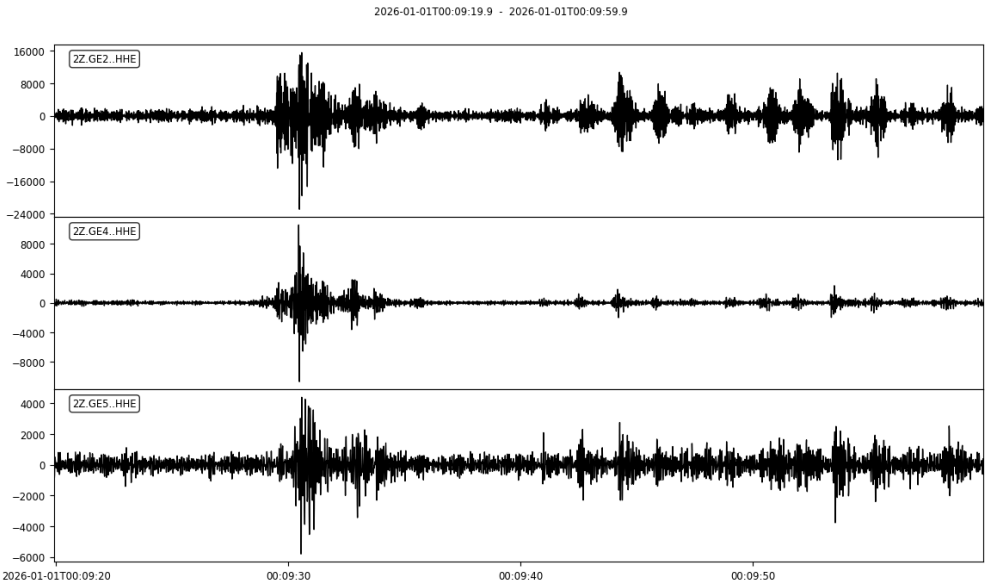
    event_window.trim(
        starttime=eruption_time - 10,
        endtime=eruption_time + 30
    )

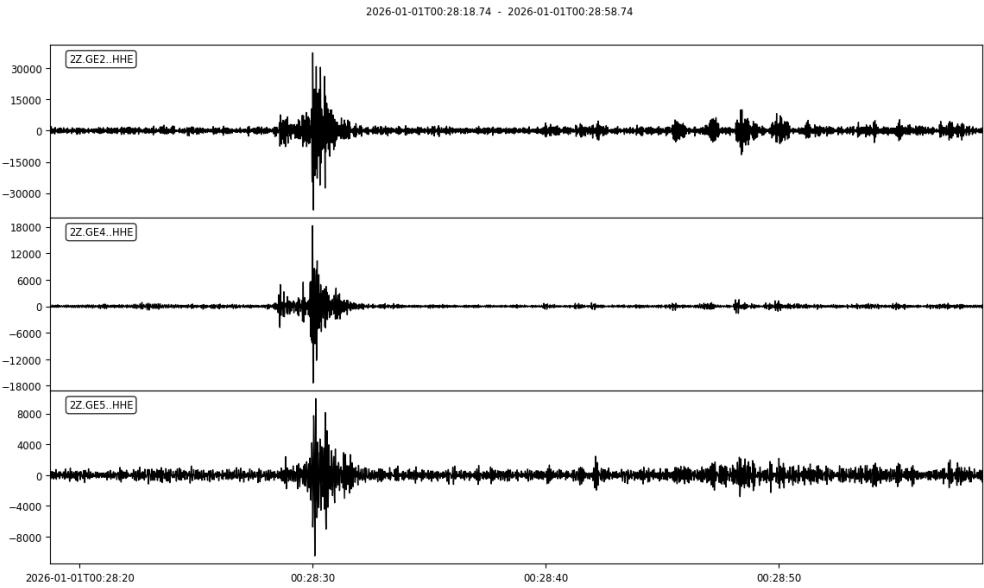
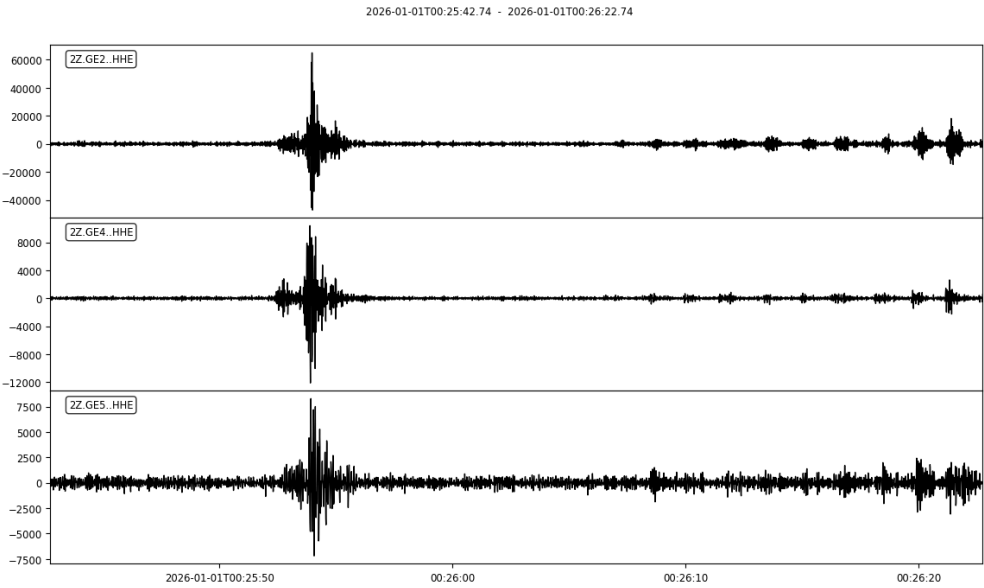
    validation_windows.append(event_window)
```

```
In [48]: for i, event_window in enumerate(
          validation_windows,
          start=1
        ):

          event_window.plot(
              size=(1200, 700),
              equal_scale=False,
              title=f"Probable Eruption {i}"
          )
```







```
In [49]: import pandas as pd

catalog_rows = []

for i, eruption in enumerate(
    preliminary_eruptions,
    start=1
):

    if i == 1:
        interval_minutes = np.nan
    else:
        interval_minutes = (
            eruption["time"]
            - preliminary_eruptions[i - 2]["time"]
        ) / 60

    catalog_rows.append({
        "Event": i,
        "Time": str(eruption["time"]),
        "Pulses": eruption["number_of_pulses"],
        "Duration_s": eruption["group_duration"],
        "Interval_min": interval_minutes
    })

catalog_30min = pd.DataFrame(catalog_rows)

catalog_30min
```

Out[49]:

	Event	Time	Pulses	Duration_s	Interval_min
0	1	2026-01-01T00:01:09.890000Z	1	3.74	NaN
1	2	2026-01-01T00:05:59.260000Z	1	3.55	4.822833
2	3	2026-01-01T00:09:29.900000Z	1	5.07	3.510667
3	4	2026-01-01T00:13:59.210000Z	1	3.58	4.488500
4	5	2026-01-01T00:18:30.450000Z	2	20.39	4.520667
5	6	2026-01-01T00:25:52.740000Z	1	3.82	7.371500
6	7	2026-01-01T00:28:28.740000Z	1	3.91	2.600000

Preliminary interpretation

Coincidence STA/LTA detection identified seven probable eruption episodes during the first 30 minutes. Most recurrence intervals are approximately 3.5–4.8 minutes, indicating a recurring seismic source. One longer interval of 7.37 minutes may indicate a missed event, while the short interval of 2.60 minutes may represent either natural variability, a secondary eruption, or a false classification. Consequently, the candidates must be visually validated before being accepted into the final eruption catalog.

Week-long Coincidence Detection

The complete seven-day dataset is processed day by day. For each day, the HHE component from stations GE2, GE4, and GE5 is preprocessed and analyzed using coincidence STA/LTA detection. Only detections recorded by all three stations are retained as probable geyser-related seismic events.

```
In [44]: from obspy.signal.trigger import coincidence_trigger

all_week_events = []

stations = ["GE2", "GE4", "GE5"]

for day_index in range(7):

    print(f"Processing day {day_index + 1}...")

    day_stream = obspy.Stream()

    for station in stations:

        station_traces = stream.select(
            station=station,
            channel="HHE"
        )

        station_traces.sort()

        trace_day = station_traces[day_index].copy()

        trace_day.detrend("demean")
        trace_day.detrend("linear")

        trace_day.taper(
            type="hann",
            max_percentage=0.05,
            max_length=10
        )

        trace_day.filter(
            "bandpass",
            freqmin=10,
            freqmax=40,
            corners=4,
            zerophase=True
        )

        day_stream += trace_day

    coincidence_events_day = coincidence_trigger(
        trigger_type="classicstalta",
        thr_on=4,
        thr_off=1.5,
        stream=day_stream,
        thr_coincidence_sum=3,
        sta=2.0,
        lta=20.0,
        details=True
    )
```

```
print(
    f"Three-station events detected: "
    f"{len(coincidence_events_day)}"
)

for candidate in coincidence_events_day:

    all_week_events.append({
        "day": day_index + 1,
        "time": candidate["time"],
        "duration": candidate["duration"],
        "stations": candidate["stations"]
    })
```

```
Processing day 1...
Three-station events detected: 398
Processing day 2...
Three-station events detected: 460
Processing day 3...
Three-station events detected: 571
Processing day 4...
Three-station events detected: 562
Processing day 5...
Three-station events detected: 576
Processing day 6...
Three-station events detected: 566
Processing day 7...
Three-station events detected: 208
```

```
In [45]: print(
    f"Total probable events during the week: "
    f"{len(all_week_events)}"
)
```

```
Total probable events during the week: 3341
```

```
In [46]: catalog_week = pd.DataFrame(all_week_events)

catalog_week["time"] = pd.to_datetime(
    catalog_week["time"].astype(str)
)

catalog_week
```

Out[46]:

	day	time	duration	stations
0	1	2026-01-01 00:01:09.890000+00:00	3.74	[GE4, GE5, GE2]
1	1	2026-01-01 00:05:59.260000+00:00	3.55	[GE4, GE2, GE5]
2	1	2026-01-01 00:09:29.900000+00:00	5.07	[GE2, GE4, GE5]
3	1	2026-01-01 00:13:59.210000+00:00	3.58	[GE2, GE4, GE5]
4	1	2026-01-01 00:18:30.450000+00:00	3.67	[GE4, GE2, GE5]
...	...	...	...	...
3336	7	2026-01-07 10:59:54.910000+00:00	4.05	[GE5, GE2, GE4]
3337	7	2026-01-07 11:01:43.260000+00:00	6.10	[GE5, GE4, GE2]
3338	7	2026-01-07 11:05:25.070000+00:00	3.73	[GE4, GE2, GE5]
3339	7	2026-01-07 11:05:45.810000+00:00	3.24	[GE4, GE2, GE5]
3340	7	2026-01-07 11:05:55.470000+00:00	2.73	[GE4, GE2, GE5]

3341 rows × 4 columns

```
In [47]: catalog_week.insert(
          0,
          "event_number",
          range(1, len(catalog_week) + 1)
        )

catalog_week.head()
```

Out[47]:

	event_number	day	time	duration	stations
0	1	1	2026-01-01 00:01:09.890000+00:00	3.74	[GE4, GE5, GE2]
1	2	1	2026-01-01 00:05:59.260000+00:00	3.55	[GE4, GE2, GE5]
2	3	1	2026-01-01 00:09:29.900000+00:00	5.07	[GE2, GE4, GE5]
3	4	1	2026-01-01 00:13:59.210000+00:00	3.58	[GE2, GE4, GE5]
4	5	1	2026-01-01 00:18:30.450000+00:00	3.67	[GE4, GE2, GE5]

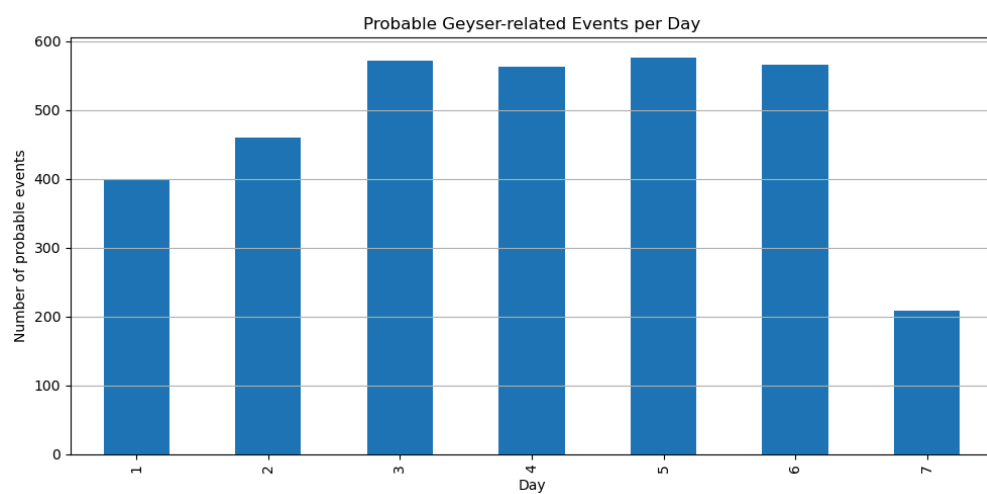
```
In [48]: catalog_week = catalog_week.sort_values(
          "time"
        ).reset_index(drop=True)

catalog_week["interval_minutes"] = (
    catalog_week["time"]
    .diff()
    .dt.total_seconds()
    / 60
)
```

```
In [49]: events_per_day = (  
    catalog_week  
    .groupby("day")  
    .size()  
    )  
  
print(events_per_day)
```

```
day  
1    398  
2    460  
3    571  
4    562  
5    576  
6    566  
7    208  
dtype: int64
```

```
In [50]: plt.figure(figsize=(10, 5))  
  
events_per_day.plot(  
    kind="bar"  
    )  
  
plt.xlabel("Day")  
plt.ylabel("Number of probable events")  
plt.title("Probable Geyser-related Events per Day")  
plt.grid(axis="y")  
  
plt.tight_layout()  
plt.show()
```

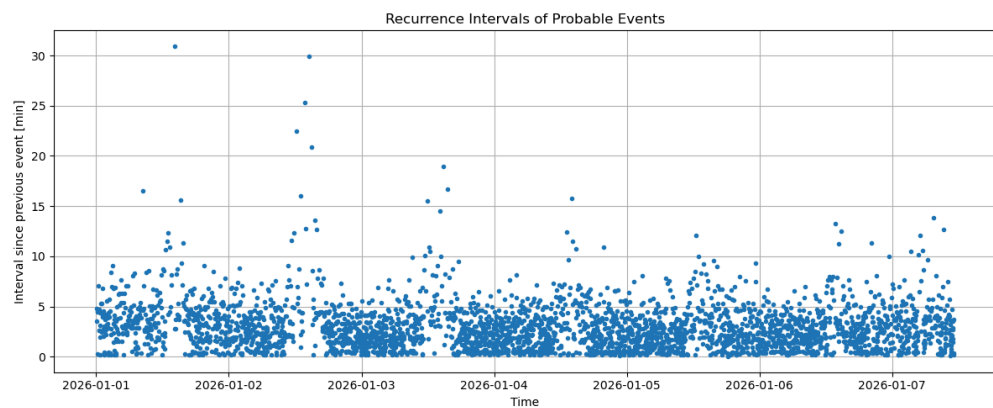


```
In [51]: plt.figure(figsize=(12, 5))

plt.plot(
    catalog_week["time"],
    catalog_week["interval_minutes"],
    marker="o",
    linestyle="none",
    markersize=3
)

plt.xlabel("Time")
plt.ylabel("Interval since previous event [min]")
plt.title("Recurrence Intervals of Probable Events")
plt.grid()

plt.tight_layout()
plt.show()
```



```
In [52]: short_intervals = catalog_week[
        catalog_week["interval_minutes"] < 2
    ]

    long_intervals = catalog_week[
        catalog_week["interval_minutes"] > 10
    ]

    print("Short intervals:")
    display(short_intervals)

    print("Long intervals:")
    display(long_intervals)
```

Short intervals:

	event_number	day	time	duration	stations
5	6	1	2026-01-01 00:18:48.440000+00:00	2.40	[GE4, GE2, GE5]
10	11	1	2026-01-01 00:37:41.500000+00:00	4.10	[GE4, GE2, GE5]
12	13	1	2026-01-01 00:42:11.520000+00:00	3.37	[GE4, GE5, GE2]
23	24	1	2026-01-01 01:22:58.110000+00:00	3.26	[GE4, GE2, GE5]
24	25	1	2026-01-01 01:23:26.130000+00:00	2.70	[GE4, GE2, GE5]
...	...	...	...	...	...
3330	3331	7	2026-01-07 10:42:31.710000+00:00	3.15	[GE4, GE2, GE5]
3334	3335	7	2026-01-07 10:54:22.710000+00:00	7.62	[GE5, GE4, GE2]
3337	3338	7	2026-01-07 11:01:43.260000+00:00	6.10	[GE5, GE4, GE2]

	event_number	day	time	duration	stations
3339	3340	7	2026-01-07 11:05:45.810000+00:00	3.24	[GE4, GE2, GE5]
3340	3341	7	2026-01-07 11:05:55.470000+00:00	2.73	[GE4, GE2, GE5]

1429 rows × 6 columns

Long intervals:

	event_number	day	time	duration	stations
144	145	1	2026-01-01 08:23:15.500000+00:00	5.15	[GE4, GE2, GE5]
213	214	1	2026-01-01 12:36:05.710000+00:00	9.62	[GE4, GE2, GE5]
214	215	1	2026-01-01 12:47:36.490000+00:00	2.77	[GE4, GE2, GE5]
215	216	1	2026-01-01 12:59:58.660000+00:00	2.50	[GE4, GE2, GE5]
217	218	1	2026-01-01 13:15:04.420000+00:00	3.02	[GE4, GE2, GE5]
222	223	1	2026-01-01 14:10:43.780000+00:00	4.20	[GE4, GE5, GE2]
233	234	1	2026-01-01 15:13:42.820000+00:00	7.42	[GE4, GE5, GE2]
236	237	1	2026-01-01 15:41:30.400000+00:00	3.04	[GE4, GE2, GE5]
635	636	2	2026-01-02 11:19:12.430000+00:00	2.86	[GE4, GE2, GE5]

	event_number	day	time	duration	stations
641	642	2	2026-01-02 11:45:24.350000+00:00	4.24	[GE4, GE2, GE5]
643	644	2	2026-01-02 12:12:44.640000+00:00	3.87	[GE4, GE2, GE5]
649	650	2	2026-01-02 13:00:14.850000+00:00	3.21	[GE4, GE2, GE5]
656	657	2	2026-01-02 13:43:30.260000+00:00	4.90	[GE5, GE2, GE4]
657	658	2	2026-01-02 13:56:17.480000+00:00	5.27	[GE5, GE4, GE2]
659	660	2	2026-01-02 14:33:24.930000+00:00	7.35	[GE4, GE2, GE5]
660	661	2	2026-01-02 14:54:15.680000+00:00	3.96	[GE4, GE2, GE5]
667	668	2	2026-01-02 15:34:02.960000+00:00	3.97	[GE5, GE4, GE2]
671	672	2	2026-01-02 15:56:01.480000+00:00	6.89	[GE5, GE4, GE2]
1166	1167	3	2026-01-03 11:31:39.270000+00:00	4.61	[GE4, GE2, GE5]
1168	1169	3	2026-01-03 11:53:46.210000+00:00	3.15	[GE4, GE2, GE5]
1171	1172	3	2026-01-03 12:10:59.090000+00:00	3.03	[GE4, GE2, GE5]
1172	1173	3	2026-01-03 12:21:28.450000+00:00	3.47	[GE4, GE2, GE5]

	event_number	day	time	duration	stations
1192	1193	3	2026-01-03 14:07:21.010000+00:00	3.63	[GE4, GE2, GE5]
1197	1198	3	2026-01-03 14:51:55.190000+00:00	3.17	[GE4, GE2, GE5]
1203	1204	3	2026-01-03 15:28:59.210000+00:00	2.70	[GE4, GE2, GE5]
1743	1744	4	2026-01-04 13:03:23.480000+00:00	5.41	[GE2, GE4, GE5]
1754	1755	4	2026-01-04 13:57:40.140000+00:00	3.40	[GE4, GE5, GE2]
1755	1756	4	2026-01-04 14:09:10.820000+00:00	3.42	[GE4, GE2, GE5]
1764	1765	4	2026-01-04 14:44:12.180000+00:00	4.67	[GE4, GE2, GE5]
1877	1878	4	2026-01-04 19:44:43.520000+00:00	4.46	[GE4, GE2, GE5]
2319	2320	5	2026-01-05 12:31:15.950000+00:00	3.43	[GE4, GE2, GE5]
2912	2913	6	2026-01-06 13:38:33.600000+00:00	2.79	[GE4, GE2, GE5]
2921	2922	6	2026-01-06 14:20:13.780000+00:00	3.13	[GE4, GE2, GE5]
2925	2926	6	2026-01-06 14:41:10.350000+00:00	3.17	[GE4, GE2, GE5]
3053	3054	6	2026-01-06 20:11:58.240000+00:00	3.11	[GE4, GE5, GE2]

	event_number	day	time	duration	stations
3122	3123	6	2026-01-06 23:25:51.030000+00:00	4.31	[GE4, GE2, GE5]
3197	3198	7	2026-01-07 03:22:09.590000+00:00	3.68	[GE4, GE2, GE5]
3226	3227	7	2026-01-07 04:46:04.010000+00:00	3.21	[GE4, GE2, GE5]
3228	3229	7	2026-01-07 05:04:35.730000+00:00	3.80	[GE4, GE2, GE5]
3231	3232	7	2026-01-07 05:25:13.200000+00:00	3.43	[GE4, GE2, GE5]
3265	3266	7	2026-01-07 07:32:03.640000+00:00	3.78	[GE4, GE2, GE5]
3298	3299	7	2026-01-07 09:21:22.450000+00:00	3.25	[GE4, GE2, GE5]

```
In [53]: catalog_week = catalog_week.sort_values(
        "time"
    ).reset_index(drop=True)

    episode_gap_seconds = 30

    episodes = []
    current_episode = []

    for _, row in catalog_week.iterrows():

        if len(current_episode) == 0:
            current_episode.append(row)
            continue

        previous_time = current_episode[-1]["time"]
        current_time = row["time"]

        gap_seconds = (
            current_time - previous_time
        ).total_seconds()

        if gap_seconds <= episode_gap_seconds:
            current_episode.append(row)

        else:
            episodes.append(current_episode)
            current_episode = [row]

    # Save final episode
    if len(current_episode) > 0:
        episodes.append(current_episode)

    print(f"Transient detections: {len(catalog_week)}")
    print(f"Grouped seismic episodes: {len(episodes)}")
```

```
Transient detections: 3341
Grouped seismic episodes: 2904
```

```
In [54]: episode_rows = []

for episode_number, episode in enumerate(
    episodes,
    start=1
):

    start_time = episode[0]["time"]

    final_detection = episode[-1]

    end_time = (
        final_detection["time"]
        + pd.to_timedelta(
            final_detection["duration"],
            unit="s"
        )
    )

    episode_rows.append({
        "episode_number": episode_number,
        "day": episode[0]["day"],
        "start_time": start_time,
        "end_time": end_time,
        "episode_duration_s": (
            end_time - start_time
        ).total_seconds(),
        "number_of_pulses": len(episode)
    })

episode_catalog = pd.DataFrame(episode_rows)

episode_catalog["interval_minutes"] = (
    episode_catalog["start_time"]
    .diff()
    .dt.total_seconds()
    / 60
)

episode_catalog.head(20)
```

Out[54]:

	episode_number	day	start_time	end_time
0	1	1	2026-01-01 00:01:09.890000+00:00	2026-01-01 00:01:13.630000248+00:00
1	2	1	2026-01-01 00:05:59.260000+00:00	2026-01-01 00:06:02.809999952+00:00
2	3	1	2026-01-01 00:09:29.900000+00:00	2026-01-01 00:09:34.969999695+00:00

	episode_number	day	start_time	end_time
3	4	1	2026-01-01 00:13:59.210000+00:00	2026-01-01 00:14:02.790000401+00:00
4	5	1	2026-01-01 00:18:30.450000+00:00	2026-01-01 00:18:50.839999857+00:00
5	6	1	2026-01-01 00:25:52.740000+00:00	2026-01-01 00:25:56.559999933+00:00
6	7	1	2026-01-01 00:28:28.740000+00:00	2026-01-01 00:28:32.649999847+00:00
7	8	1	2026-01-01 00:33:11.130000+00:00	2026-01-01 00:33:14.430000191+00:00
8	9	1	2026-01-01 00:35:46.550000+00:00	2026-01-01 00:35:50.459999847+00:00
9	10	1	2026-01-01 00:37:41.500000+00:00	2026-01-01 00:37:45.599999905+00:00
10	11	1	2026-01-01 00:42:01.760000+00:00	2026-01-01 00:42:14.889999886+00:00
11	12	1	2026-01-01 00:45:17.950000+00:00	2026-01-01 00:45:21.689999771+00:00
12	13	1	2026-01-01 00:50:38.020000+00:00	2026-01-01 00:50:41.299999971+00:00
13	14	1	2026-01-01 00:55:54.700000+00:00	2026-01-01 00:55:58.880000067+00:00
14	15	1	2026-01-01 00:58:21.600000+00:00	2026-01-01 00:58:24.820000029+00:00
15	16	1	2026-01-01 01:05:08.590000+00:00	2026-01-01 01:05:11.569999781+00:00
16	17	1	2026-01-01 01:09:14.500000+00:00	2026-01-01 01:09:18.240000010+00:00
17	18	1	2026-01-01 01:13:13.920000+00:00	2026-01-01 01:13:17.119999809+00:00
18	19	1	2026-01-01 01:15:16.400000+00:00	2026-01-01 01:15:19.350000048+00:00
19	20	1	2026-01-01 01:19:50.750000+00:00	2026-01-01 01:19:55.439999819+00:00

```
In [55]: print(  
    episode_catalog[  
        "interval_minutes"  
    ].describe()  
)
```

```
count    2903.000000  
mean      3.205048  
std       2.291808  
min       0.500667  
25%      1.682917  
50%      2.764667  
75%      4.094750  
max      30.942833  
Name: interval_minutes, dtype: float64
```

```
In [56]: print(  
    "Episodes shorter than 2 minutes apart:",  
    (  
        episode_catalog["interval_minutes"] < 2  
    ).sum()  
)
```

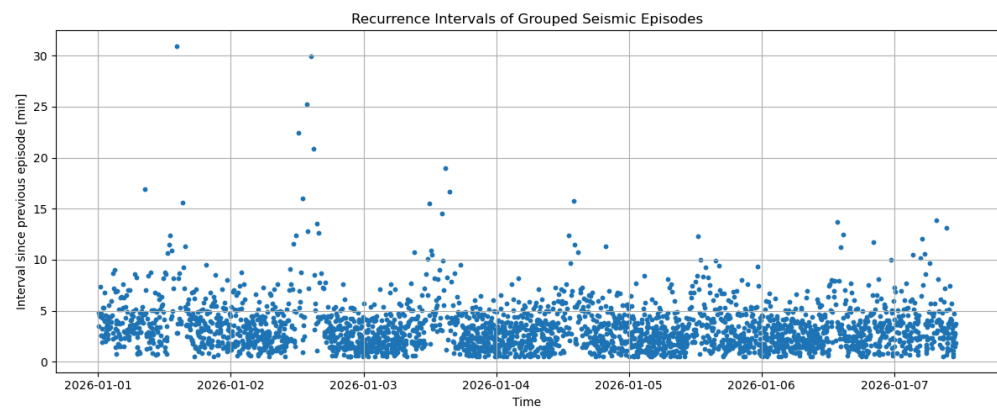
```
Episodes shorter than 2 minutes apart: 958
```

```
In [57]: plt.figure(figsize=(12, 5))

plt.scatter(
    episode_catalog["start_time"],
    episode_catalog["interval_minutes"],
    s=10
)

plt.xlabel("Time")
plt.ylabel("Interval since previous episode [min]")
plt.title("Recurrence Intervals of Grouped Seismic Episodes")
plt.grid()

plt.tight_layout()
plt.show()
```



```
In [58]: recording_hours = {}

for day_index in range(7):

    trace_day = (
        stream.select(
            station="GE2",
            channel="HHE"
        )
        .copy()
    )

    trace_day.sort()

    trace = trace_day[day_index]

    duration_hours = (
        trace.stats.endtime
        - trace.stats.starttime
    ) / 3600

    recording_hours[day_index + 1] = duration_hours

print(recording_hours)
```

```
{1: 23.999997222222223, 2: 23.999997222222223, 3: 23.999997222222223,
4: 23.999997222222223, 5: 23.999997222222223, 6: 23.999997222222223,
7: 12.148919444444445}
```

```
In [59]: episodes_per_day = (
    episode_catalog
    .groupby("day")
    .size()
)

episode_rate = pd.Series({
    day: episodes_per_day.get(day, 0)
    / recording_hours[day]

    for day in recording_hours
})

print(episode_rate)
```

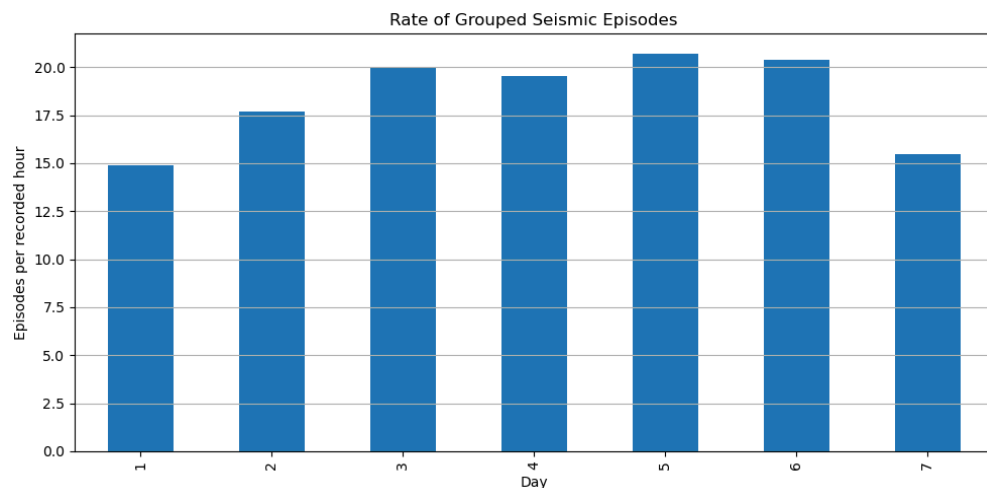
```
1    14.875002
2    17.666669
3    20.000002
4    19.541669
5    20.708336
6    20.375002
7    15.474627
dtype: float64
```

```
In [60]: plt.figure(figsize=(10, 5))

episode_rate.plot(kind="bar")

plt.xlabel("Day")
plt.ylabel("Episodes per recorded hour")
plt.title("Rate of Grouped Seismic Episodes")
plt.grid(axis="y")

plt.tight_layout()
plt.show()
```



Coincidence STA/LTA detection produced 3341 transient detections recorded by all three stations. The large number of short recurrence intervals indicated that many detections represented individual pulses within broader seismic episodes rather than separate eruptions. Consequently, detections separated by no more than 30 seconds were grouped into common episodes before further classification.

```
In [61]: network_amplitudes = []

for _, episode in episode_catalog.iterrows():

    episode_time = episode["start_time"]

    amplitudes = []

    for station in ["GE2", "GE4", "GE5"]:

        station_traces = stream.select(
            station=station,
            channel="HHE"
        ).copy()

        station_traces.sort()

        day_index = int(episode["day"]) - 1
        trace = station_traces[day_index].copy()

        short_window = trace.copy()

        start_time = episode_time - pd.Timedelta(seconds=2)
        end_time = episode_time + pd.Timedelta(seconds=10)

        short_window.trim(
            starttime=UTCDateTime(start_time.to_pydatetime()),
            endtime=UTCDateTime(end_time.to_pydatetime())
        )

        short_window.detrend("demean")
        short_window.detrend("linear")

        short_window.taper(
            type="hann",
            max_percentage=0.05
        )

        short_window.filter(
            "bandpass",
            freqmin=10,
            freqmax=40,
            corners=4,
            zerophase=True
        )

        peak_amplitude = np.max(
            np.abs(short_window.data)
        )

        amplitudes.append(peak_amplitude)

network_amplitudes.append(
```

```
        np.mean(amplitudes)
    )

episode_catalog["network_peak_amplitude"] = network_amplitudes
```

Conservative Selection of Principal Eruption Candidates

To reduce the influence of closely spaced secondary seismic activity, episodes occurring within 120 seconds were grouped, and the episode with the highest mean network peak amplitude was retained. This produces a conservative catalogue of principal candidates. Because this additional criterion may also remove genuine closely spaced eruptions, results are reported both before and after this selection.

```
In [62]: minimum_separation_seconds = 120

principal_candidates = []
current_cluster = []

for _, episode in episode_catalog.iterrows():

    if len(current_cluster) == 0:
        current_cluster.append(episode)
        continue

    previous_time = current_cluster[-1]["start_time"]
    current_time = episode["start_time"]

    gap = (
        current_time - previous_time
    ).total_seconds()

    if gap < minimum_separation_seconds:
        current_cluster.append(episode)

    else:
        strongest = max(
            current_cluster,
            key=lambda row: row["network_peak_amplitude"]
        )

        principal_candidates.append(strongest)
        current_cluster = [episode]

if len(current_cluster) > 0:

    strongest = max(
        current_cluster,
        key=lambda row: row["network_peak_amplitude"]
    )

    principal_candidates.append(strongest)
```

```
In [63]: eruption_candidates = pd.DataFrame(
    principal_candidates
).reset_index(drop=True)

eruption_candidates.insert(
    0,
    "candidate_number",
    range(1, len(eruption_candidates) + 1)
)

eruption_candidates["interval_minutes"] = (
    eruption_candidates["start_time"]
    .diff()
    .dt.total_seconds()
    / 60
)

print(
    "Principal eruption candidates:",
    len(eruption_candidates)
)

eruption_candidates.head(20)
```

Principal eruption candidates: 1946

Out[63]:

	candidate_number	episode_number	day	start_time	
0	1	1	1	2026-01-01 00:01:09.890000+00:00	2026-01-01 00:01:09.890000+00:00
1	2	2	1	2026-01-01 00:05:59.260000+00:00	2026-01-01 00:05:59.260000+00:00
2	3	3	1	2026-01-01 00:09:29.900000+00:00	2026-01-01 00:09:29.900000+00:00
3	4	4	1	2026-01-01 00:13:59.210000+00:00	2026-01-01 00:13:59.210000+00:00
4	5	5	1	2026-01-01 00:18:30.450000+00:00	2026-01-01 00:18:30.450000+00:00
5	6	6	1	2026-01-01 00:25:52.740000+00:00	2026-01-01 00:25:52.740000+00:00
6	7	7	1	2026-01-01 00:28:28.740000+00:00	2026-01-01 00:28:28.740000+00:00
7	8	8	1	2026-01-01 00:33:11.130000+00:00	2026-01-01 00:33:11.130000+00:00

	candidate_number	episode_number	day	start_time	
8	9	10	1	2026-01-01 00:37:41.500000+00:00	20 00
9	10	11	1	2026-01-01 00:42:01.760000+00:00	20 00
10	11	12	1	2026-01-01 00:45:17.950000+00:00	20 00
11	12	13	1	2026-01-01 00:50:38.020000+00:00	20 00
12	13	14	1	2026-01-01 00:55:54.700000+00:00	20 00
13	14	15	1	2026-01-01 00:58:21.600000+00:00	20 00
14	15	16	1	2026-01-01 01:05:08.590000+00:00	20 00
15	16	17	1	2026-01-01 01:09:14.500000+00:00	20 00
16	17	18	1	2026-01-01 01:13:13.920000+00:00	20 00
17	18	19	1	2026-01-01 01:15:16.400000+00:00	20 00
18	19	20	1	2026-01-01 01:19:50.750000+00:00	20 00
19	20	21	1	2026-01-01 01:22:35.580000+00:00	20 00

```
In [64]: print(
          eruption_candidates["interval_minutes"].describe()
        )

print(
    "Intervals below 2 minutes:",
    (
        eruption_candidates["interval_minutes"] < 2
    ).sum()
)
```

```
count    1945.000000
mean      4.783678
std       2.387129
min       2.001667
25%       3.353167
50%       4.269333
75%       5.587000
max       30.942833
Name: interval_minutes, dtype: float64
Intervals below 2 minutes: 0
```

```
In [65]: candidates_per_day = (
          eruption_candidates
          .groupby("day")
          .size()
        )

print(candidates_per_day)
```

```
day
1    293
2    286
3    294
4    302
5    307
6    318
7    146
dtype: int64
```

```
In [66]: candidate_rate = pd.Series({
        day: candidates_per_day.get(day, 0)
        / recording_hours[day]

        for day in recording_hours
    })

print(candidate_rate)
```

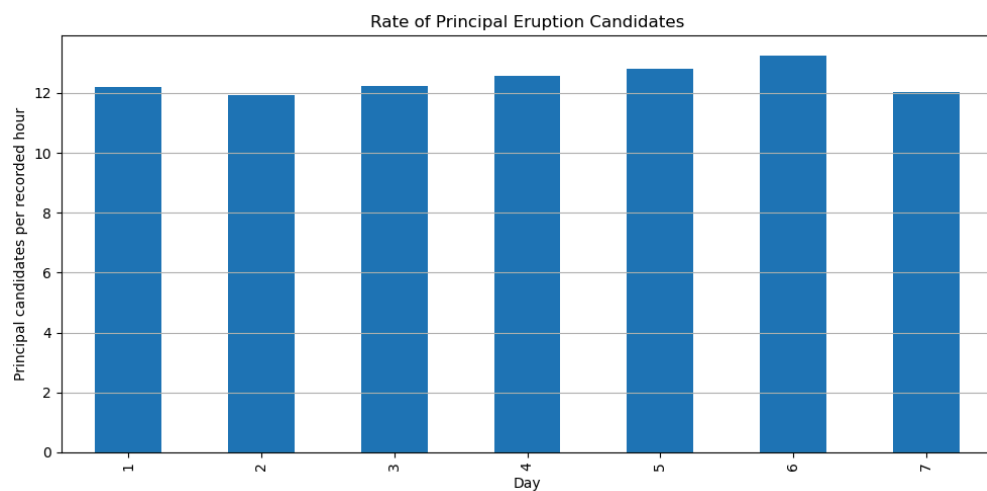
```
1    12.208335
2    11.916668
3    12.250001
4    12.583335
5    12.791668
6    13.250002
7    12.017530
dtype: float64
```

```
In [67]: plt.figure(figsize=(10, 5))

candidate_rate.plot(kind="bar")

plt.xlabel("Day")
plt.ylabel("Principal candidates per recorded hour")
plt.title("Rate of Principal Eruption Candidates")
plt.grid(axis="y")

plt.tight_layout()
plt.show()
```

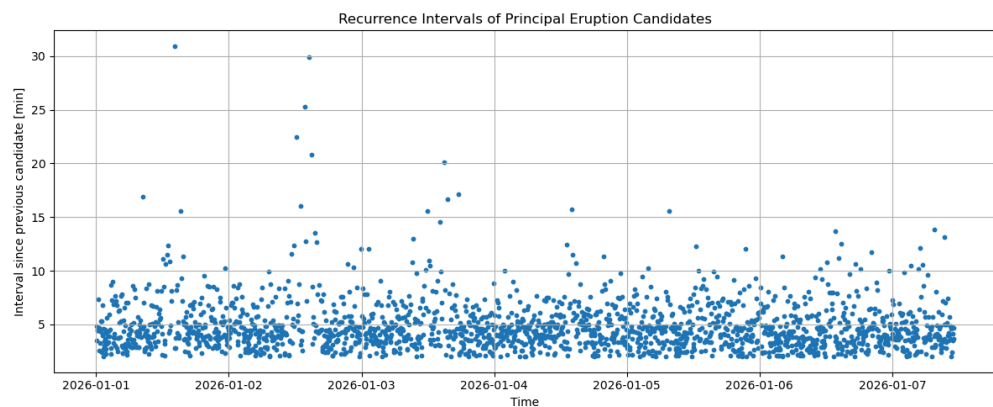


```
In [68]: plt.figure(figsize=(12, 5))

plt.scatter(
    eruption_candidates["start_time"],
    eruption_candidates["interval_minutes"],
    s=10
)

plt.xlabel("Time")
plt.ylabel("Interval since previous candidate [min]")
plt.title("Recurrence Intervals of Principal Eruption Candidates")
plt.grid()

plt.tight_layout()
plt.show()
```



Three-station coincidence STA/LTA detection identified 3341 coherent seismic transients. After grouping detections separated by no more than 30 seconds, 2904 seismic episodes remained. To reduce the influence of closely spaced secondary activity, episodes occurring within two-minute clusters were compared using their mean peak amplitude across the three stations, and the strongest episode in each cluster was retained. This produced 1946 principal eruption candidates.

```
In [69]: representative_options = eruption_candidates[
    (eruption_candidates["number_of_pulses"] == 1)
    &
    (eruption_candidates["episode_duration_s"] >= 3)
    &
    (eruption_candidates["episode_duration_s"] <= 8)
    &
    (eruption_candidates["interval_minutes"] >= 3)
    &
    (eruption_candidates["interval_minutes"] <= 8)
].copy()
```

```
In [70]: representative_options = representative_options.sort_values(
        "network_peak_amplitude",
        ascending=False
    ).reset_index(drop=True)

print(
    "Suitable representative-event options:",
    len(representative_options)
)

representative_options[
    [
        "candidate_number",
        "day",
        "start_time",
        "episode_duration_s",
        "interval_minutes",
        "network_peak_amplitude"
    ]
].head(10)
```

Suitable representative-event options: 1042

Out[70]:

	candidate_number	day	start_time	episode_duration_s
0	425	2	2026-01-02 09:50:23.670000+00:00	3.059999
1	841	3	2026-01-03 21:46:06.510000+00:00	3.359999
2	57	1	2026-01-01 04:07:52.870000+00:00	3.049999
3	834	3	2026-01-03 21:17:15.630000+00:00	3.179999
4	648	3	2026-01-03 05:17:54.140000+00:00	3.009999
5	570	2	2026-01-02 23:17:22.380000+00:00	3.189999
6	942	4	2026-01-04 05:13:35.300000+00:00	3.019999
7	306	2	2026-01-02 01:07:19.170000+00:00	3.319999
8	332	2	2026-01-02 03:12:52.640000+00:00	3.009999

	candidate_number	day	start_time	episode_duration_s
9	537	2	2026-01-02 20:34:03.590000+00:00	3.179999

```

In [71]: top_options = representative_options.head(5)

for option_index, candidate in top_options.iterrows():

    candidate_time = UTCDateTime(
        candidate["start_time"].to_pydatetime()
    )

    candidate_day = int(candidate["day"])

    candidate_stream = obspy.Stream()

    for station in ["GE2", "GE4", "GE5"]:

        station_traces = stream.select(
            station=station,
            channel="HHE"
        ).copy()

        station_traces.sort()

        candidate_trace = station_traces[
            candidate_day - 1
        ].copy()

        candidate_trace.trim(
            starttime=candidate_time - 10,
            endtime=candidate_time + 30
        )

        candidate_trace.detrend("demean")
        candidate_trace.detrend("linear")

        candidate_trace.taper(
            type="hann",
            max_percentage=0.05
        )

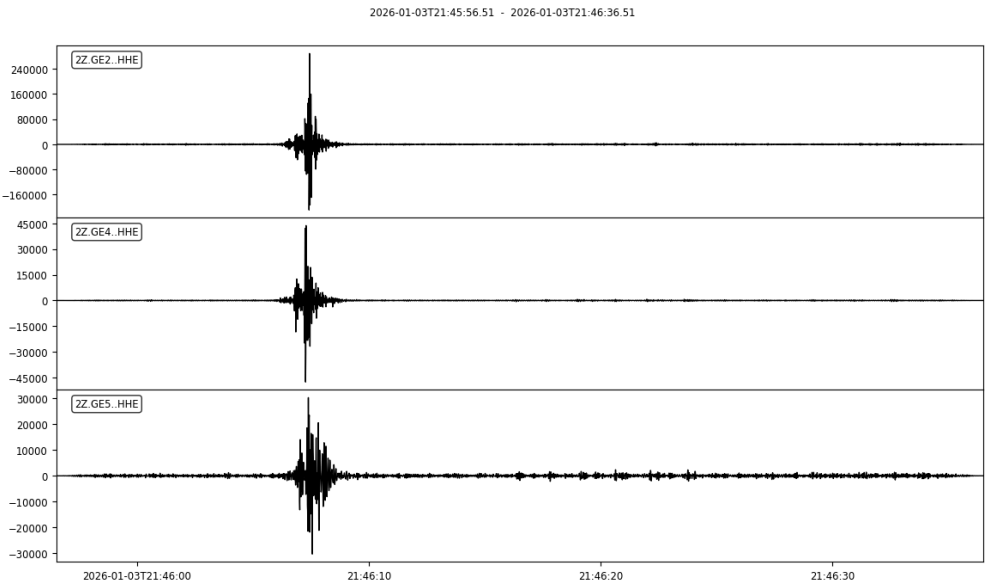
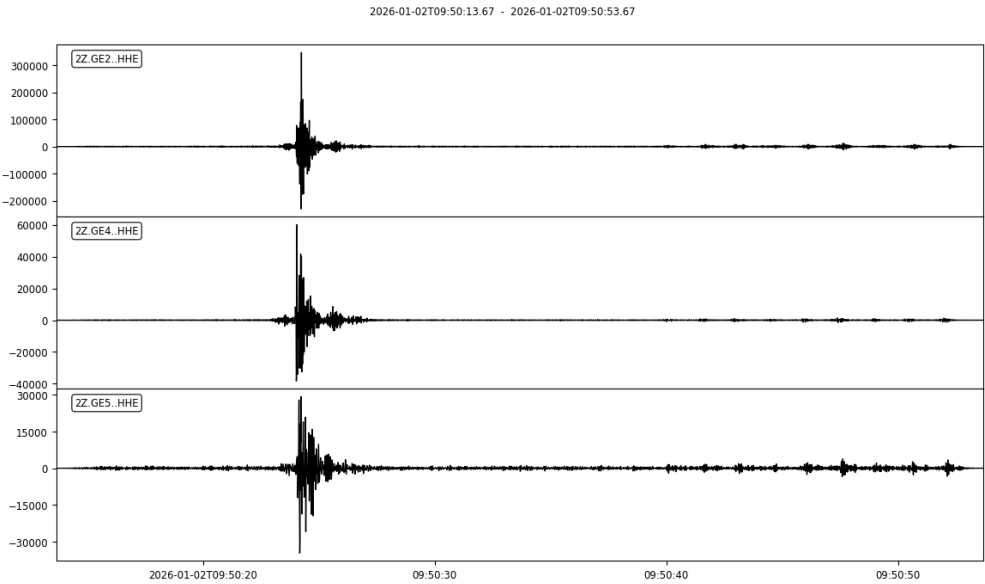
        candidate_trace.filter(
            "bandpass",
            freqmin=10,
            freqmax=40,
            corners=4,
            zerophase=True
        )

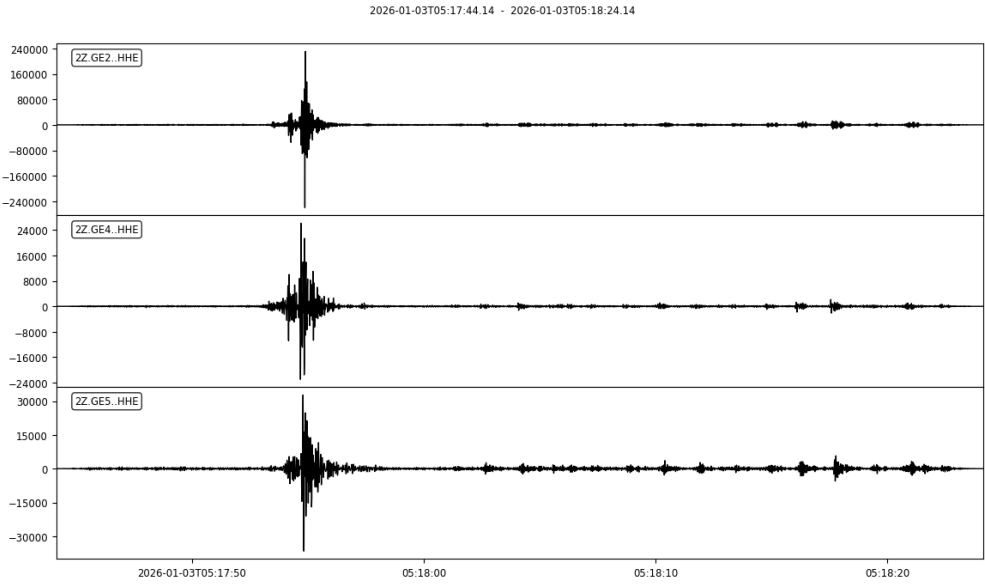
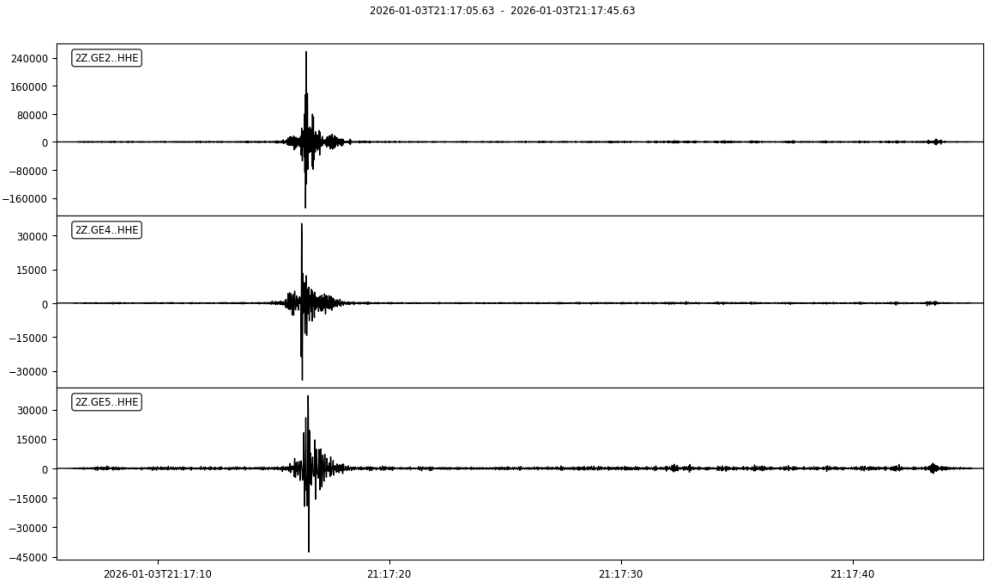
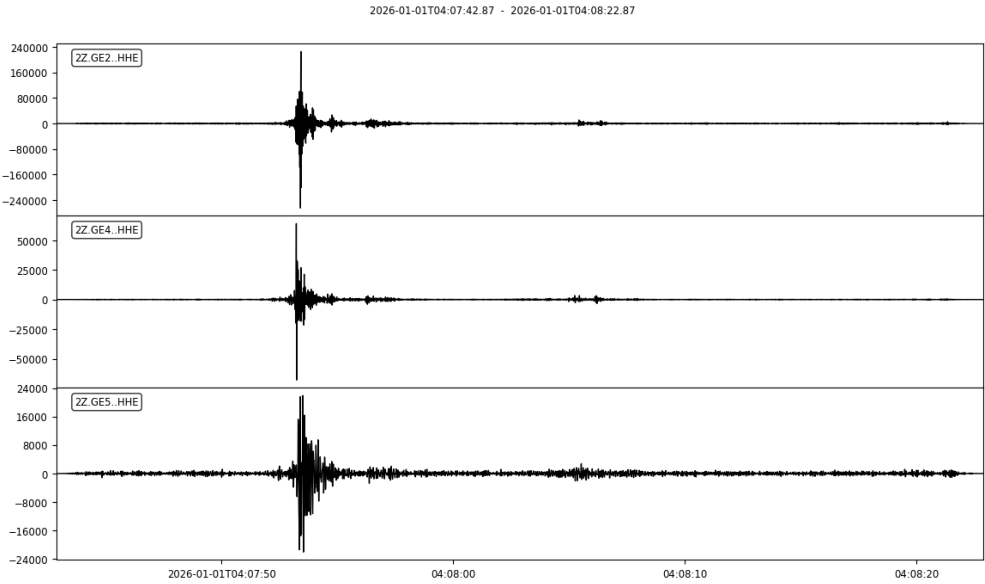
        candidate_stream += candidate_trace

    candidate_stream.plot(
        size=(1200, 700),
        equal_scale=False,
        title=(
            f"Representative Option {option_index + 1}: "

```

```
f"{candidate['start_time']}"  
)  
)
```





Detailed Analysis of the Representative Event

A strong, isolated event recorded on 2 January 2026 at approximately 09:50:23.67 UTC was selected for detailed analysis. The event was clearly recorded at all three stations and exhibited a high signal-to-noise ratio, a distinct onset, and a decaying waveform.

The three components from station GE2 are used for seismogram, spectrogram, particle-motion, and horizontal analysis.

```
In [72]: representative_time = UTCDateTime(  
         "2026-01-02T09:50:23.670000Z"  
         )  
  
         representative_time
```

```
Out[72]: 2026-01-02T09:50:23.670000Z
```

```

In [73]: representative_stream = obspy.Stream()

for channel in ["HHE", "HHN", "HHZ"]:

    channel_traces = stream.select(
        station="GE2",
        channel=channel
    ).copy()

    channel_traces.sort()

    # Day 2 has index 1
    trace_component = channel_traces[1].copy()

    trace_component.trim(
        starttime=representative_time - 5,
        endtime=representative_time + 15
    )

    trace_component.detrend("demean")
    trace_component.detrend("linear")

    trace_component.taper(
        type="hann",
        max_percentage=0.05
    )

    trace_component.filter(
        "bandpass",
        freqmin=10,
        freqmax=40,
        corners=4,
        zerophase=True
    )

    representative_stream += trace_component

print(representative_stream)

```

```

3 Trace(s) in Stream:
2Z.GE2..HHE | 2026-01-02T09:50:18.670000Z - 2026-01-
02T09:50:38.670000Z | 100.0 Hz, 2001 samples
2Z.GE2..HHN | 2026-01-02T09:50:18.670000Z - 2026-01-
02T09:50:38.670000Z | 100.0 Hz, 2001 samples
2Z.GE2..HHZ | 2026-01-02T09:50:18.670000Z - 2026-01-
02T09:50:38.670000Z | 100.0 Hz, 2001 samples

```

```

In [74]: fig, axes = plt.subplots(
        3,
        1,
        figsize=(13, 8),
        sharex=True
    )

    channel_names = {
        "HHE": "East-West",
        "HHN": "North-South",
        "HHZ": "Vertical"
    }

    for ax, trace_component in zip(
        axes,
        representative_stream
    ):

        relative_time = (
            trace_component.times()
            - 5
        )

        ax.plot(
            relative_time,
            trace_component.data,
            linewidth=0.7
        )

        channel = trace_component.stats.channel

        ax.set_ylabel(
            f"{channel}\nAmplitude [counts]"
        )

        ax.text(
            0.98,
            0.85,
            channel_names[channel],
            transform=ax.transAxes,
            horizontalalignment="right"
        )

        ax.axvline(
            0,
            linestyle="--",
            linewidth=1,
            label="Detected onset"
        )

        ax.grid()

    axes[0].set_title(

```

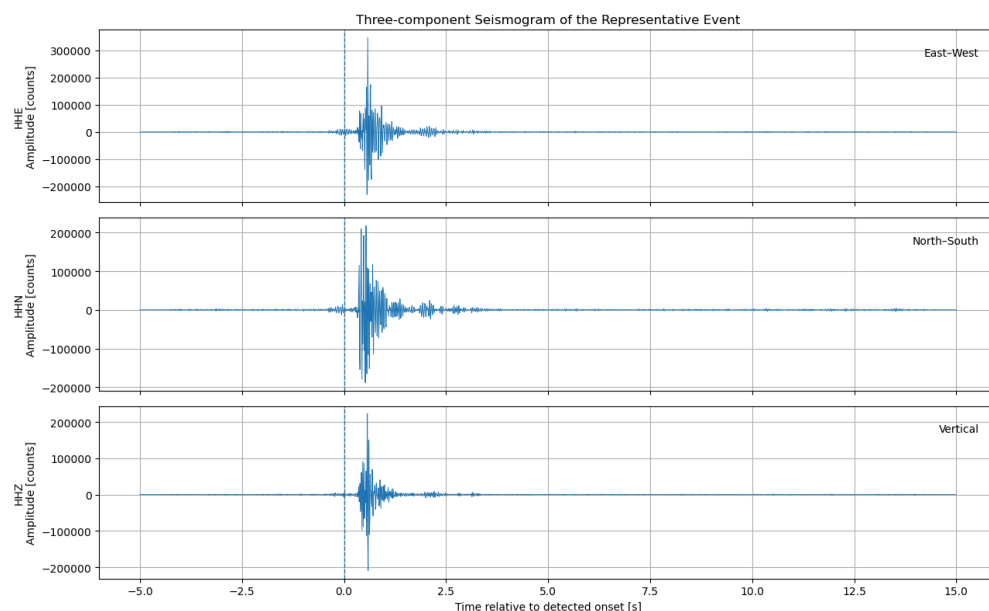
```

    "Three-component Seismogram of the Representative Event"
)

axes[-1].set_xlabel(
    "Time relative to detected onset [s]"
)

plt.tight_layout()
plt.show()

```



In []:

Three-component seismogram of the representative event recorded at station GE2. The traces were demeaned, linearly detrended, tapered, and band-pass filtered between 10 and 40 Hz. Time is shown relative to the automatically detected event onset.

```

In
[107]: # Select the HHE component
hhe = representative_stream.select(channel="HHE")[0].copy()

# Trim to a shorter window around the detected onset
plot_start = representative_time - 1
plot_end = representative_time + 4

hhe_short = hhe.copy().trim(
    starttime=plot_start,
    endtime=plot_end
)

# Relative time: onset = 0 s
relative_time = (
    hhe_short.times()
    + (hhe_short.stats.starttime - representative_time)
)

fig, axes = plt.subplots(
    2,
    1,
    figsize=(11, 7),
    sharex=True
)

# Waveform
axes[0].plot(
    relative_time,
    hhe_short.data,
    linewidth=0.8
)

axes[0].axvline(
    0,
    linestyle="--",
    linewidth=1,
    label="Detected onset"
)

axes[0].set_ylabel("Amplitude [counts]")
axes[0].set_title(
    "Representative Event at GE2 - HHE Component"
)
axes[0].grid()
axes[0].legend()

# Spectrogram
axes[1].specgram(
    hhe_short.data,
    NFFT=64,
    Fs=hhe_short.stats.sampling_rate,
    noverlap=56,
    cmap=plt.cm.gist_heat,

```

```

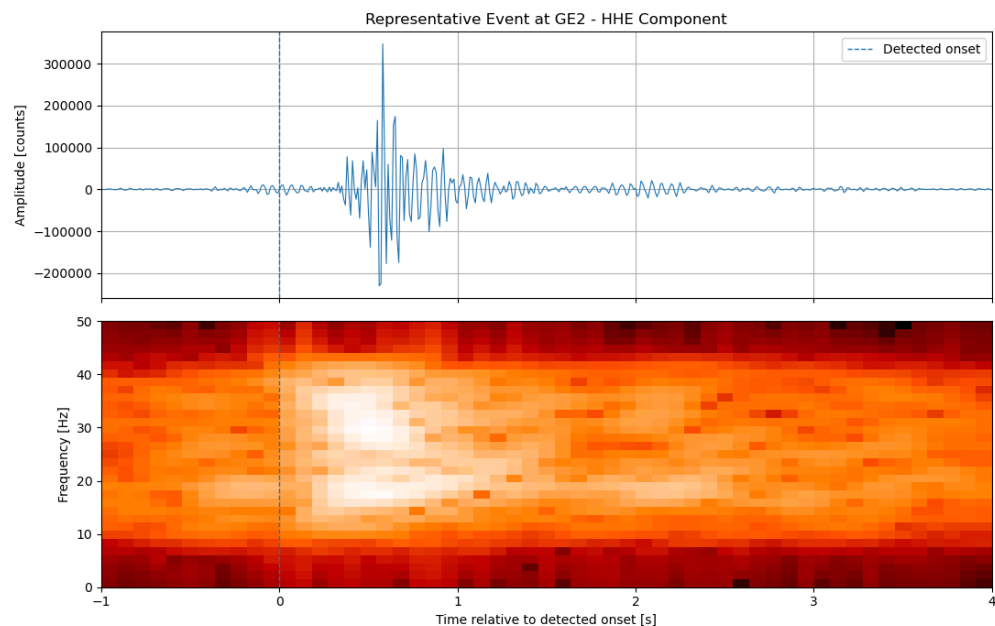
xextent=(-1, 4)
)

axes[1].axvline(
    0,
    linestyle="--",
    linewidth=1
)

axes[1].set_xlabel(
    "Time relative to detected onset [s]"
)
axes[1].set_ylabel("Frequency [Hz]")
axes[1].set_ylim(0, 50)
axes[1].set_xlim(-1, 4)

plt.tight_layout()
plt.show()

```



```

In [76]: east = representative_stream.select(
           channel="HHE"
         )[0]

north = representative_stream.select(
        channel="HHN"
      )[0]

vertical = representative_stream.select(
        channel="HHZ"
      )[0]

```

```

In [77]: # Use a short window immediately after the detected onset
motion_start = representative_time + 0.10
motion_end = representative_time + 0.40

east_motion = representative_stream.select(channel="HHE")[0].copy()
north_motion = representative_stream.select(channel="HHN")[0].copy()
vertical_motion = representative_stream.select(channel="HHZ")[0].copy()

east_motion.trim(
    starttime=motion_start,
    endtime=motion_end
)

north_motion.trim(
    starttime=motion_start,
    endtime=motion_end
)

vertical_motion.trim(
    starttime=motion_start,
    endtime=motion_end
)

# Remove the local mean before particle-motion analysis
east_motion.detrend("demean")
north_motion.detrend("demean")
vertical_motion.detrend("demean")

print(
    "Particle-motion window:",
    motion_start,
    "to",
    motion_end
)

print(
    "Samples:",
    len(east_motion.data),
    len(north_motion.data),
    len(vertical_motion.data)
)

```

```

Particle-motion window: 2026-01-02T09:50:23.770000Z to 2026-01-
02T09:50:24.070000Z
Samples: 31 31 31

```

```
In [78]: print(  
        len(east_motion.data),  
        len(north_motion.data),  
        len(vertical_motion.data)  
        )
```

31 31 31

```
In [79]: fig, axes = plt.subplots(
        1,
        3,
        figsize=(15, 5)
    )

    axes[0].plot(
        east_motion.data,
        north_motion.data,
        linewidth=0.6
    )

    axes[0].set_xlabel("East-West amplitude [counts]")
    axes[0].set_ylabel("North-South amplitude [counts]")
    axes[0].set_title("Horizontal Particle Motion")
    axes[0].grid()
    axes[0].axis("equal")

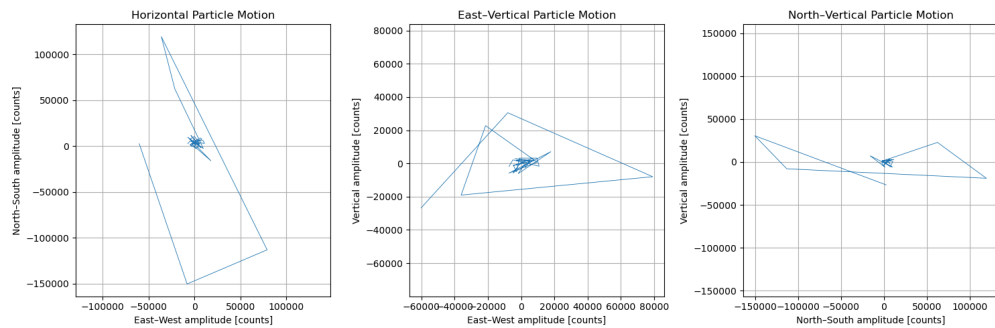
    axes[1].plot(
        east_motion.data,
        vertical_motion.data,
        linewidth=0.6
    )

    axes[1].set_xlabel("East-West amplitude [counts]")
    axes[1].set_ylabel("Vertical amplitude [counts]")
    axes[1].set_title("East-Vertical Particle Motion")
    axes[1].grid()
    axes[1].axis("equal")

    axes[2].plot(
        north_motion.data,
        vertical_motion.data,
        linewidth=0.6
    )

    axes[2].set_xlabel("North-South amplitude [counts]")
    axes[2].set_ylabel("Vertical amplitude [counts]")
    axes[2].set_title("North-Vertical Particle Motion")
    axes[2].grid()
    axes[2].axis("equal")

    plt.tight_layout()
    plt.show()
```



```
In [80]: motion_start = representative_time + 0.10
motion_end = representative_time + 0.40
```

```
east_motion = east.copy()
north_motion = north.copy()
vertical_motion = vertical.copy()

east_motion.trim(
    starttime=motion_start,
    endtime=motion_end
)

north_motion.trim(
    starttime=motion_start,
    endtime=motion_end
)

vertical_motion.trim(
    starttime=motion_start,
    endtime=motion_end
)

east_motion.detrend("demean")
north_motion.detrend("demean")
vertical_motion.detrend("demean")

print(
    len(east_motion.data),
    len(north_motion.data),
    len(vertical_motion.data)
)
```

```
31 31 31
```

```

In [81]: fig, axes = plt.subplots(
            1,
            3,
            figsize=(15, 5)
        )

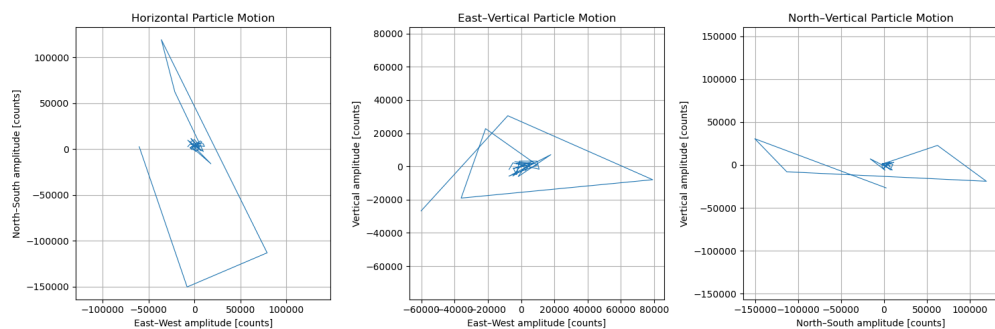
axes[0].plot(
    east_motion.data,
    north_motion.data,
    linewidth=0.8
)
axes[0].set_xlabel("East-West amplitude [counts]")
axes[0].set_ylabel("North-South amplitude [counts]")
axes[0].set_title("Horizontal Particle Motion")
axes[0].axis("equal")
axes[0].grid()

axes[1].plot(
    east_motion.data,
    vertical_motion.data,
    linewidth=0.8
)
axes[1].set_xlabel("East-West amplitude [counts]")
axes[1].set_ylabel("Vertical amplitude [counts]")
axes[1].set_title("East-Vertical Particle Motion")
axes[1].axis("equal")
axes[1].grid()

axes[2].plot(
    north_motion.data,
    vertical_motion.data,
    linewidth=0.8
)
axes[2].set_xlabel("North-South amplitude [counts]")
axes[2].set_ylabel("Vertical amplitude [counts]")
axes[2].set_title("North-Vertical Particle Motion")
axes[2].axis("equal")
axes[2].grid()

plt.tight_layout()
plt.show()

```



```

In [82]: east_data = east_motion.data.astype(float)
north_data = north_motion.data.astype(float)

# Remove any remaining means
east_data = east_data - np.mean(east_data)
north_data = north_data - np.mean(north_data)

# Rows are samples; columns are East and North
horizontal_data = np.column_stack(
    (east_data, north_data)
)

# Covariance matrix
covariance_matrix = np.cov(
    horizontal_data,
    rowvar=False
)

# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eigh(
    covariance_matrix
)

# Eigenvector corresponding to greatest variance
principal_index = np.argmax(eigenvalues)
principal_vector = eigenvectors[:, principal_index]

east_direction = principal_vector[0]
north_direction = principal_vector[1]

# Azimuth clockwise from north
azimuth = np.degrees(
    np.arctan2(
        east_direction,
        north_direction
    )
) % 360

opposite_azimuth = (
    azimuth + 180
) % 360

print(f"Principal orientation: {azimuth:.1f}°")
print(f"Opposite direction:    {opposite_azimuth:.1f}°")

```

```

Principal orientation: 343.2°
Opposite direction:    163.2°

```

```
In [83]: eigenvalues_sorted = np.sort(
          eigenvalues
        )[::-1]

        rectilinearity = (
            1
            - eigenvalues_sorted[1]
            / eigenvalues_sorted[0]
        )

        print(
            f"Horizontal rectilinearity: "
            f"{rectilinearity:.3f}"
        )
```

Horizontal rectilinearity: 0.853

```
In [84]: east_full = east.data.astype(float)
north_full = north.data.astype(float)

sampling_rate = east.stats.sampling_rate

window_length_seconds = 0.30
step_seconds = 0.05

window_samples = int(
    window_length_seconds * sampling_rate
)

step_samples = int(
    step_seconds * sampling_rate
)

# The representative trace begins 5 seconds before the onset
onset_index = int(5 * sampling_rate)

search_start = onset_index
search_end = onset_index + int(2 * sampling_rate)

window_results = []

for start_index in range(
    search_start,
    search_end - window_samples + 1,
    step_samples
):

    end_index = start_index + window_samples

    east_window = east_full[
        start_index:end_index
    ]

    north_window = north_full[
        start_index:end_index
    ]

    east_window = east_window - np.mean(east_window)
    north_window = north_window - np.mean(north_window)

    horizontal_window = np.column_stack(
        (east_window, north_window)
    )

    covariance = np.cov(
        horizontal_window,
        rowvar=False
    )

    eigenvalues, eigenvectors = np.linalg.eigh(
```

```

        covariance
    )

    order = np.argsort(eigenvalues)[::-1]

    eigenvalues = eigenvalues[order]
    eigenvectors = eigenvectors[:, order]

    if eigenvalues[0] == 0:
        continue

    rectilinearity = (
        1 - eigenvalues[1] / eigenvalues[0]
    )

    principal_vector = eigenvectors[:, 0]

    east_direction = principal_vector[0]
    north_direction = principal_vector[1]

    azimuth = np.degrees(
        np.arctan2(
            east_direction,
            north_direction
        )
    ) % 360

    relative_start = (
        start_index - onset_index
    ) / sampling_rate

    relative_end = (
        end_index - onset_index
    ) / sampling_rate

    window_results.append({
        "start_s": relative_start,
        "end_s": relative_end,
        "rectilinearity": rectilinearity,
        "azimuth": azimuth,
        "opposite_azimuth": (
            azimuth + 180
        ) % 360
    })

```

```
In [85]: polarization_results = pd.DataFrame(
        window_results
    )

    polarization_results = (
        polarization_results
        .sort_values(
            "rectilinearity",
            ascending=False
        )
        .reset_index(drop=True)
    )

    polarization_results.head(10)
```

Out[85]:

	start_s	end_s	rectilinearity	azimuth	opposite_azimuth
0	0.10	0.40	0.909317	344.492997	164.492997
1	0.15	0.45	0.874457	355.313981	175.313981
2	0.20	0.50	0.855648	354.702261	174.702261
3	1.70	2.00	0.852390	39.586782	219.586782
4	1.65	1.95	0.848169	41.331738	221.331738
5	1.60	1.90	0.823098	231.619525	51.619525
6	0.25	0.55	0.798266	352.690638	172.690638
7	1.55	1.85	0.740190	229.912897	49.912897
8	0.00	0.30	0.727059	257.965725	77.965725
9	1.50	1.80	0.674276	225.561893	45.561893

```
In [86]: plt.figure(figsize=(11, 5))

plt.plot(
    polarization_results
    .sort_values("start_s")["start_s"],
    polarization_results
    .sort_values("start_s")["rectilinearity"],
    marker="o"
)

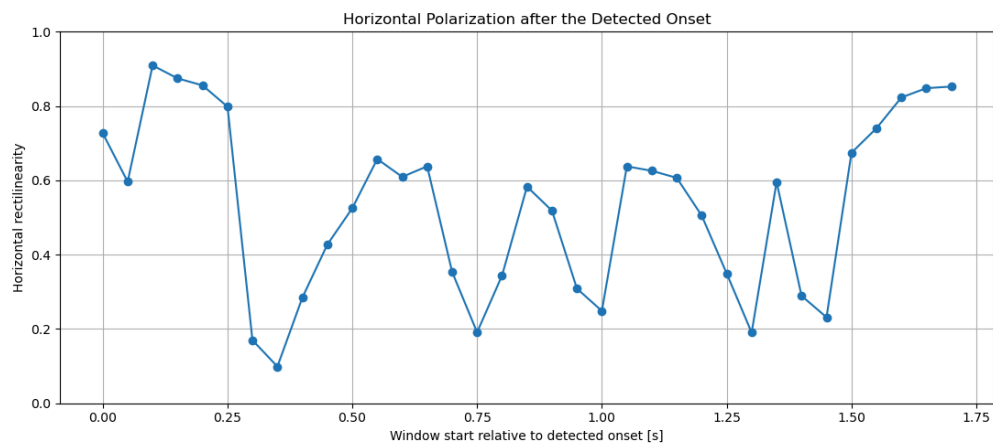
plt.xlabel(
    "Window start relative to detected onset [s]"
)

plt.ylabel("Horizontal rectilinearity")

plt.title(
    "Horizontal Polarization after the Detected Onset"
)

plt.ylim(0, 1)
plt.grid()

plt.tight_layout()
plt.show()
```



```
In [87]: best_window = polarization_results.iloc[0]

print(
    f"Best window: "
    f"{best_window['start_s']:.2f} to "
    f"{best_window['end_s']:.2f} s"
)

print(
    f"Rectilinearity: "
    f"{best_window['rectilinearity']:.3f}"
)

print(
    f"Orientation: "
    f"{best_window['azimuth']:.1f}° "
    f"or "
    f"{best_window['opposite_azimuth']:.1f}°"
)
```

```
Best window: 0.10 to 0.40 s
Rectilinearity: 0.909
Orientation: 344.5° or 164.5°
```

A moving-window polarization analysis was applied to the first two seconds after the detected onset. The highest horizontal rectilinearity, 0.909, occurred between 0.10 and 0.40 s. Principal-component analysis yielded a horizontal orientation of 344.5°/164.5°, with an inherent 180° ambiguity.

```

In
[105]: best_start_index = onset_index + int(
        best_window["start_s"] * sampling_rate
    )

    best_end_index = onset_index + int(
        best_window["end_s"] * sampling_rate
    )

    best_east = east_full[best_start_index:best_end_index].copy()
    best_north = north_full[best_start_index:best_end_index].copy()

    best_east_centered = best_east - np.mean(best_east)
    best_north_centered = best_north - np.mean(best_north)

    azimuth_rad = np.radians(float(best_window["azimuth"]))

    east_direction = np.sin(azimuth_rad)
    north_direction = np.cos(azimuth_rad)

    axis_length = max(
        np.max(np.abs(best_east_centered)),
        np.max(np.abs(best_north_centered))
    )

    plt.figure(figsize=(7, 7))

    plt.plot(
        best_east_centered,
        best_north_centered,
        linewidth=1.2,
        label="Particle motion"
    )

    plt.plot(
        [
            -axis_length * east_direction,
            axis_length * east_direction
        ],
        [
            -axis_length * north_direction,
            axis_length * north_direction
        ],
        linestyle="--",
        linewidth=2,
        label=(
            f"Polarization axis: "
            f"{best_window['azimuth']:.1f} deg / "
            f"{best_window['opposite_azimuth']:.1f} deg"
        )
    )

    plt.xlabel("East-West amplitude [counts]")
    plt.ylabel("North-South amplitude [counts]")

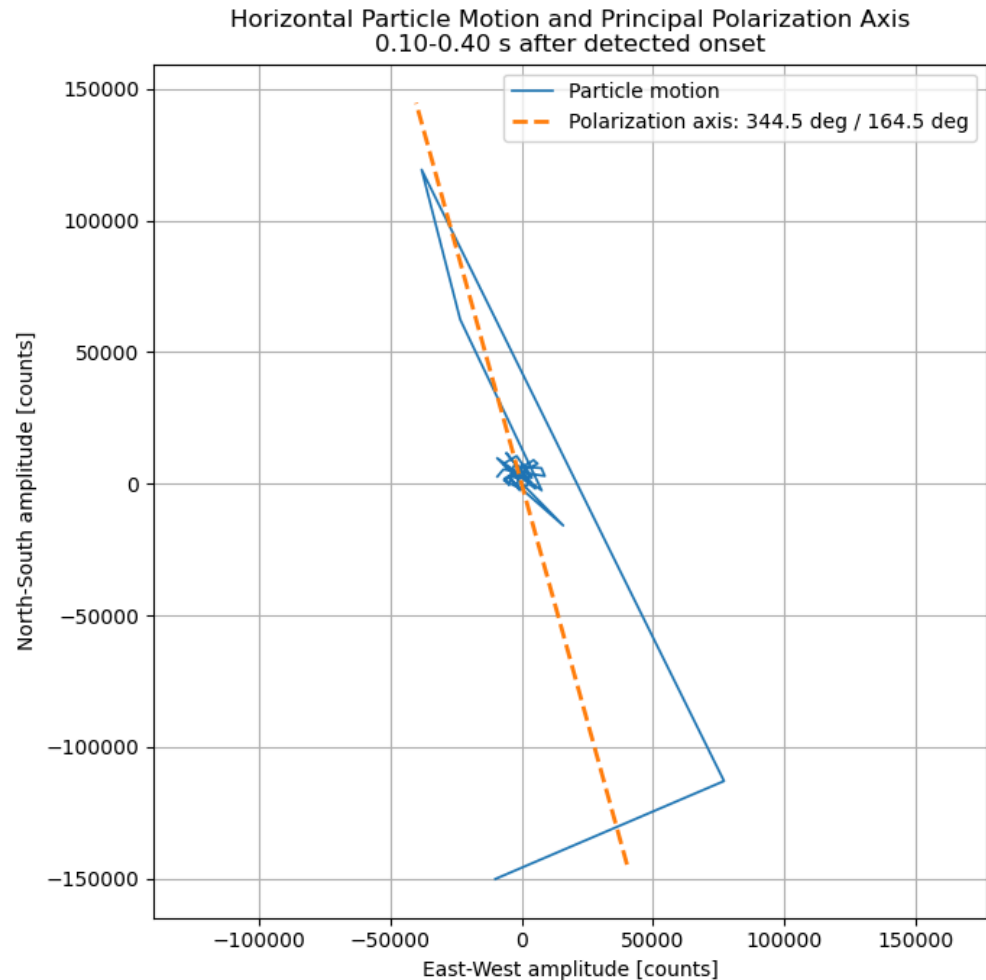
```

```

plt.title(
    "Horizontal Particle Motion and Principal Polarization Axis\n"
    f"{best_window['start_s']:.2f}-"
    f"{best_window['end_s']:.2f} s after detected onset"
)

plt.axis("equal")
plt.grid()
plt.legend()
plt.tight_layout()
plt.show()

```



The horizontal particle motion between 0.10 and 0.40 s after the detected onset was strongly rectilinear, with a rectilinearity of 0.909 and a dominant horizontal polarization axis of $344.5^\circ/164.5^\circ$. Because polarization defines an axis rather than a unique direction, the sourceward direction cannot be determined from this analysis alone. The result applies specifically to the selected short window and not to the complete event.

the result applies specifically to the selected short window, not to the complete event.

```
In [90]: import pandas as pd  
import numpy as np
```

```

In [91]: # Moving-window horizontal polarization analysis

east_full = east.data.astype(float)
north_full = north.data.astype(float)

sampling_rate = east.stats.sampling_rate

window_length_seconds = 0.30
step_seconds = 0.05

window_samples = int(window_length_seconds * sampling_rate)
step_samples = int(step_seconds * sampling_rate)

# representative_stream begins 5 seconds before the detected onset
onset_index = int(5 * sampling_rate)

# Examine the first 2 seconds after the detected onset
search_start = onset_index
search_end = onset_index + int(2 * sampling_rate)

window_results = []

for start_index in range(
    search_start,
    search_end - window_samples + 1,
    step_samples
):
    end_index = start_index + window_samples

    east_window = east_full[start_index:end_index].copy()
    north_window = north_full[start_index:end_index].copy()

    # Remove the mean inside each window
    east_window -= np.mean(east_window)
    north_window -= np.mean(north_window)

    horizontal_window = np.column_stack(
        (east_window, north_window)
    )

    covariance = np.cov(
        horizontal_window,
        rowvar=False
    )

    eigenvalues, eigenvectors = np.linalg.eigh(covariance)

    # Largest eigenvalue first
    order = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[order]
    eigenvectors = eigenvectors[:, order]

    if eigenvalues[0] <= 0:

```

```

        continue

    rectilinearity = 1 - (
        eigenvalues[1] / eigenvalues[0]
    )

    principal_vector = eigenvectors[:, 0]

    east_direction = principal_vector[0]
    north_direction = principal_vector[1]

    # Clockwise from north
    azimuth = (
        np.degrees(
            np.arctan2(
                east_direction,
                north_direction
            )
        )
        + 360
    ) % 360

    relative_start = (
        start_index - onset_index
    ) / sampling_rate

    relative_end = (
        end_index - onset_index
    ) / sampling_rate

    window_results.append({
        "start_s": relative_start,
        "end_s": relative_end,
        "rectilinearity": rectilinearity,
        "azimuth": azimuth,
        "opposite_azimuth": (azimuth + 180) % 360
    })

polarization_results = pd.DataFrame(window_results)

polarization_results = (
    polarization_results
    .sort_values(
        "rectilinearity",
        ascending=False
    )
    .reset_index(drop=True)
)

polarization_results.head(10)

```

Out[91]:

	start_s	end_s	rectilinearity	azimuth	opposite_azimuth
0	0.10	0.40	0.909317	344.492997	164.492997
1	0.15	0.45	0.874457	355.313981	175.313981
2	0.20	0.50	0.855648	354.702261	174.702261
3	1.70	2.00	0.852390	39.586782	219.586782
4	1.65	1.95	0.848169	41.331738	221.331738
5	1.60	1.90	0.823098	231.619525	51.619525
6	0.25	0.55	0.798266	352.690638	172.690638
7	1.55	1.85	0.740190	229.912897	49.912897
8	0.00	0.30	0.727059	257.965725	77.965725
9	1.50	1.80	0.674276	225.561893	45.561893

```

In [92]: best_window = polarization_results.iloc[0]

print(
    f"Best window: "
    f"{best_window['start_s']:.2f} to "
    f"{best_window['end_s']:.2f} s"
)

print(
    f"Rectilinearity: "
    f"{best_window['rectilinearity']:.3f}"
)

print(
    f"Orientation: "
    f"{best_window['azimuth']:.1f}° or "
    f"{best_window['opposite_azimuth']:.1f}°"
)

```

```

Best window: 0.10 to 0.40 s
Rectilinearity: 0.909
Orientation: 344.5° or 164.5°

```

```
In [93]: polarization_summary = {
    "representative_event": str(representative_time),
    "station": "GE2",
    "window_start_s": float(best_window["start_s"]),
    "window_end_s": float(best_window["end_s"]),
    "rectilinearity": float(best_window["rectilinearity"]),
    "orientation_deg": float(best_window["azimuth"]),
    "opposite_orientation_deg": float(
        best_window["opposite_azimuth"]
    ),
    "ambiguity_resolved": False
}

polarization_summary
```

```
Out[93]: {'representative_event': '2026-01-02T09:50:23.670000Z',
'station': 'GE2',
'window_start_s': 0.1,
'window_end_s': 0.4,
'rectilinearity': 0.9093171765576638,
'orientation_deg': 344.49299667133727,
'opposite_orientation_deg': 164.49299667133732,
'ambiguity_resolved': False}
```

```
In [94]: eruption_candidates.to_csv(
    "principal_eruption_candidates.csv",
    index=False
)

print("Catalog exported.")
```

Catalog exported.

```
In [95]: summary_stats = {
    "Three-station transients": 3341,
    "Grouped seismic episodes": 2904,
    "Principal eruption candidates": 1946,
    "Representative event": str(representative_time),
    "Representative station": "GE2",
    "Filter": "10-40 Hz band-pass",
    "STA window": "2.0 s",
    "LTA window": "20.0 s",
    "Polarization window": "0.10-0.40 s",
    "Rectilinearity": 0.909,
    "Polarization orientation": "344.5° / 164.5°",
    "Diretional ambiguity resolved": False
}

for key, value in summary_stats.items():
    print(f"{key}: {value}")
```

```
Three-station transients: 3341
Grouped seismic episodes: 2904
Principal eruption candidates: 1946
Representative event: 2026-01-02T09:50:23.670000Z
Representative station: GE2
Filter: 10-40 Hz band-pass
STA window: 2.0 s
LTA window: 20.0 s
Polarization window: 0.10-0.40 s
Rectilinearity: 0.909
Polarization orientation: 344.5° / 164.5°
Diretional ambiguity resolved: False
```

```

In [96]: required_variables = [
    "catalog_week",
    "episode_catalog",
    "eruption_candidates",
    "representative_time",
    "best_window"
]

missing_variables = [
    name for name in required_variables
    if name not in globals()
]

if missing_variables:
    raise NameError(
        "Run the earlier cells that create these variables first: "
        + ", ".join(missing_variables)
    )

summary_stats = {
    "Three-station transients": len(catalog_week),
    "Grouped seismic episodes": len(episode_catalog),
    "Principal eruption candidates": len(eruption_candidates),
    "Representative event": str(representative_time),
    "Representative station": "GE2",
    "Filter": "10-40 Hz band-pass",
    "STA window": "2.0 s",
    "LTA window": "20.0 s",
    "Polarization window": (
        f"{best_window['start_s']:.2f}-"
        f"{best_window['end_s']:.2f} s"
    ),
    "Rectilinearity": round(
        float(best_window["rectilinearity"]), 3
    ),
    "Polarization orientation": (
        f"{best_window['azimuth']:.1f}° / "
        f"{best_window['opposite_azimuth']:.1f}°"
    ),
    "Directional ambiguity resolved": False
}

for key, value in summary_stats.items():
    print(f"{key}: {value}")

```

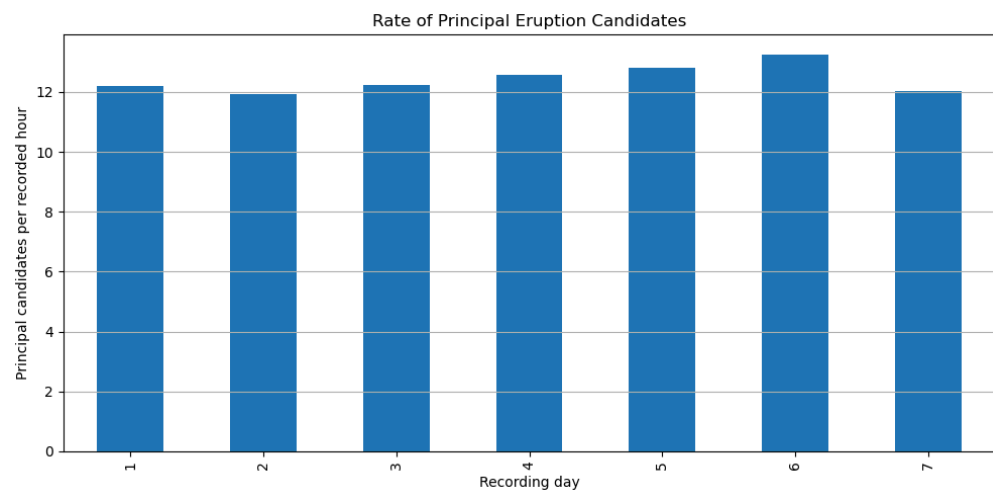
```

Three-station transients: 3341
Grouped seismic episodes: 2904
Principal eruption candidates: 1946
Representative event: 2026-01-02T09:50:23.670000Z
Representative station: GE2
Filter: 10-40 Hz band-pass
STA window: 2.0 s
LTA window: 20.0 s

```

Polarization window: 0.10–0.40 s
Rectilinearity: 0.909
Polarization orientation: 344.5° / 164.5°
Directional ambiguity resolved: False

```
In [97]: candidates_per_day = (  
    eruption_candidates  
    .groupby("day")  
    .size()  
    )  
  
candidate_rate = pd.Series({  
    day: candidates_per_day.get(day, 0)  
    / recording_hours[day]  
  
    for day in recording_hours  
    })  
  
plt.figure(figsize=(10, 5))  
  
candidate_rate.plot(kind="bar")  
  
plt.xlabel("Recording day")  
plt.ylabel("Principal candidates per recorded hour")  
plt.title("Rate of Principal Eruption Candidates")  
plt.grid(axis="y")  
  
plt.tight_layout()  
plt.show()
```



```

In [98]: valid_intervals = (
            eruption_candidates["interval_minutes"]
            .dropna()
        )

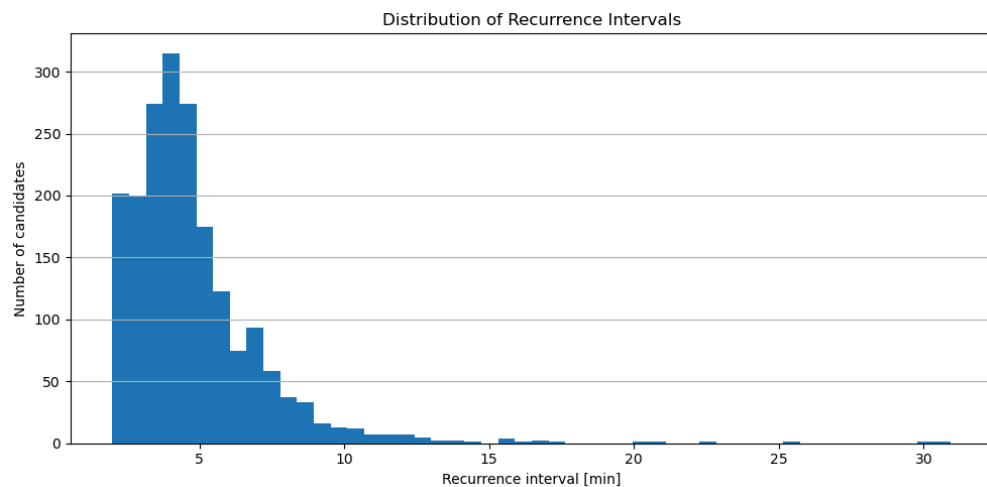
plt.figure(figsize=(10, 5))

plt.hist(
    valid_intervals,
    bins=50
)

plt.xlabel("Recurrence interval [min]")
plt.ylabel("Number of candidates")
plt.title("Distribution of Recurrence Intervals")
plt.grid(axis="y")

plt.tight_layout()
plt.show()

```



```

In [99]: print(valid_intervals.describe())

```

```

count    1945.000000
mean       4.783678
std        2.387129
min        2.001667
25%        3.353167
50%        4.269333
75%        5.587000
max       30.942833
Name: interval_minutes, dtype: float64

```

```
In [101]: from obspy import read_inventory

inventory = read_inventory(
    r"C:\Users\marwan\Downloads\2Z_Strokkur2024.xml"
)
```

```
In [102]: station_rows = []

for network in inventory:
    for station in network:
        station_rows.append({
            "station": station.code,
            "latitude": station.latitude,
            "longitude": station.longitude,
            "elevation_m": station.elevation
        })

station_coordinates = pd.DataFrame(station_rows)
station_coordinates
```

Out[102]:

	station	latitude	longitude	elevation_m
0	GE5	64.312783	-20.302133	125.9
1	GE2	64.312970	-20.301258	143.8
2	GE4	64.312499	-20.300098	117.1

Station Coordinates

The coordinates of stations GE2, GE4, and GE5 were read from the supplied StationXML metadata. The representative-event polarization analysis was performed at GE2. The horizontal components are geographically oriented, with HHN pointing north and HHE pointing east.

A separate coordinate for the Strokkur vent is not contained in the StationXML file. Therefore, the measured horizontal polarization remains an axis with two possible directions, 344.5° and 164.5°.

Conclusion

Continuous seismic recordings from stations GE2, GE4, and GE5 were analysed for the period 1–7 January 2026. After demeaning, linear detrending, tapering, and 10–40 Hz band-pass filtering, STA/LTA coincidence detection identified 3341 transients recorded at all three stations. Grouping closely related detections produced 2904 seismic episodes, from which 1946 conservative principal eruption candidates were selected.

The principal candidates showed recurrence intervals concentrated mainly between approximately 3 and 6 minutes. A representative event recorded on 2 January 2026 at 09:50:23.67 UTC displayed a clear onset and broadband energy within the selected frequency band. Horizontal particle-motion analysis at GE2 gave a highly rectilinear polarization window between 0.10 and 0.40 s after onset, with a rectilinearity of 0.909 and a polarization axis of $344.5^\circ/164.5^\circ$.

Because no independent Strokkur vent coordinate was available in the supplied StationXML metadata, the 180° directional ambiguity could not be resolved. Overall, the workflow successfully detected and characterised recurrent seismic signals associated with Strokkur activity.

Name: Marwan Karam Saeed Saad

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).